GRAFOS II

Taller de Programación Competitiva UMSS 2024

El algoritmo de Dijkstra

Definición

Dado un grafo con pesos **no negativos** y un nodo inicial s:

- El algoritmo de Dijkstra nos permite calcular el camino mínimo (más corto) desde el nodo s a los demás nodos del grafo.

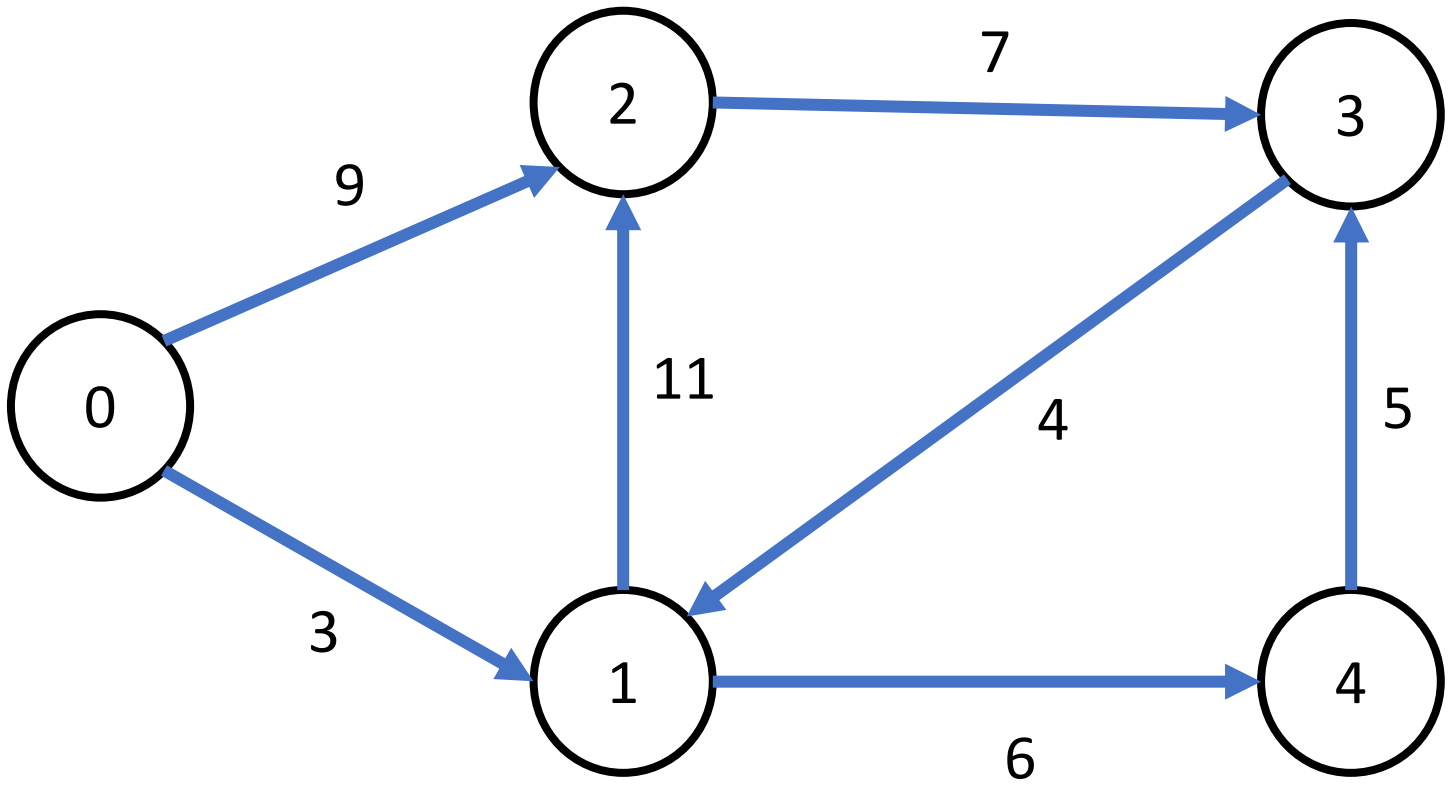
El algoritmo de Dijkstra

Algoritmo

- Sea $d[x]$ la distancia del nodo s al nodo x.
- Vamos a mantener una lista de nodos $pendientes$.
- Inicialmente la distancia a todos los nodos es infinita: $d[x] = \infty$
- Añadimos el nodo s a $pendientes$.
- La distancia al nodo s es 0. $d[s] = 0$
- Mientras $pendientes$ no esté vacío:
  - Elegimos el nodo con menor distancia de $pendientes$. Digamos que este nodo es u.
  - Retiramos el nodo u de $pendientes$.
  - Revisamos los vecinos del nodo u. Sea v un nodo vecino de u y w el costo de la arista de u a v.
  - Si $d[u] + w < d[v]$, podemos mejorar la distancia al nodo v:
    - $d[v] = d[u] + w$
    - Añadimos v a $pendientes$.

El algoritmo de Dijkstra

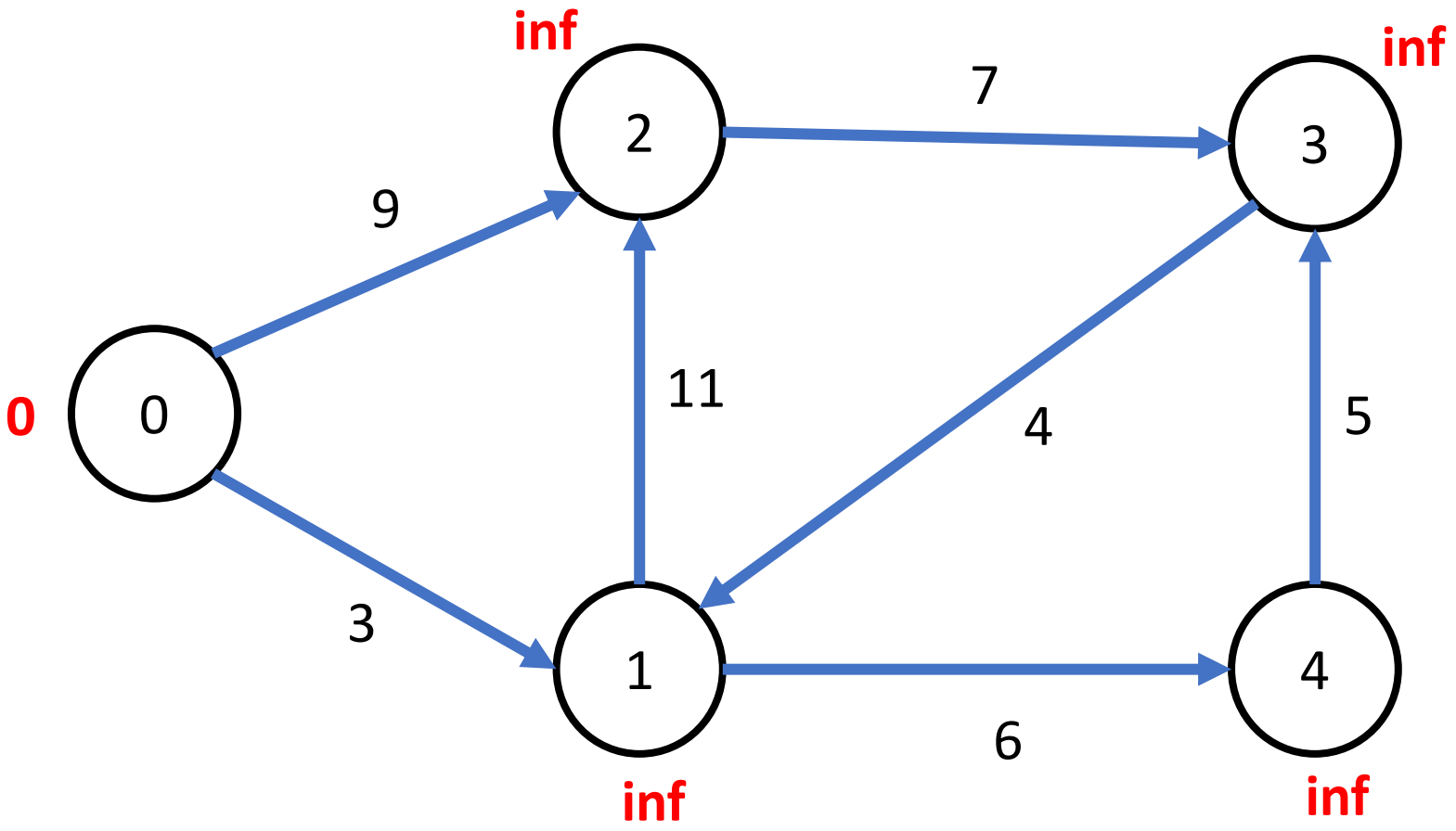
Ejemplo



El algoritmo de Dijkstra

Ejemplo

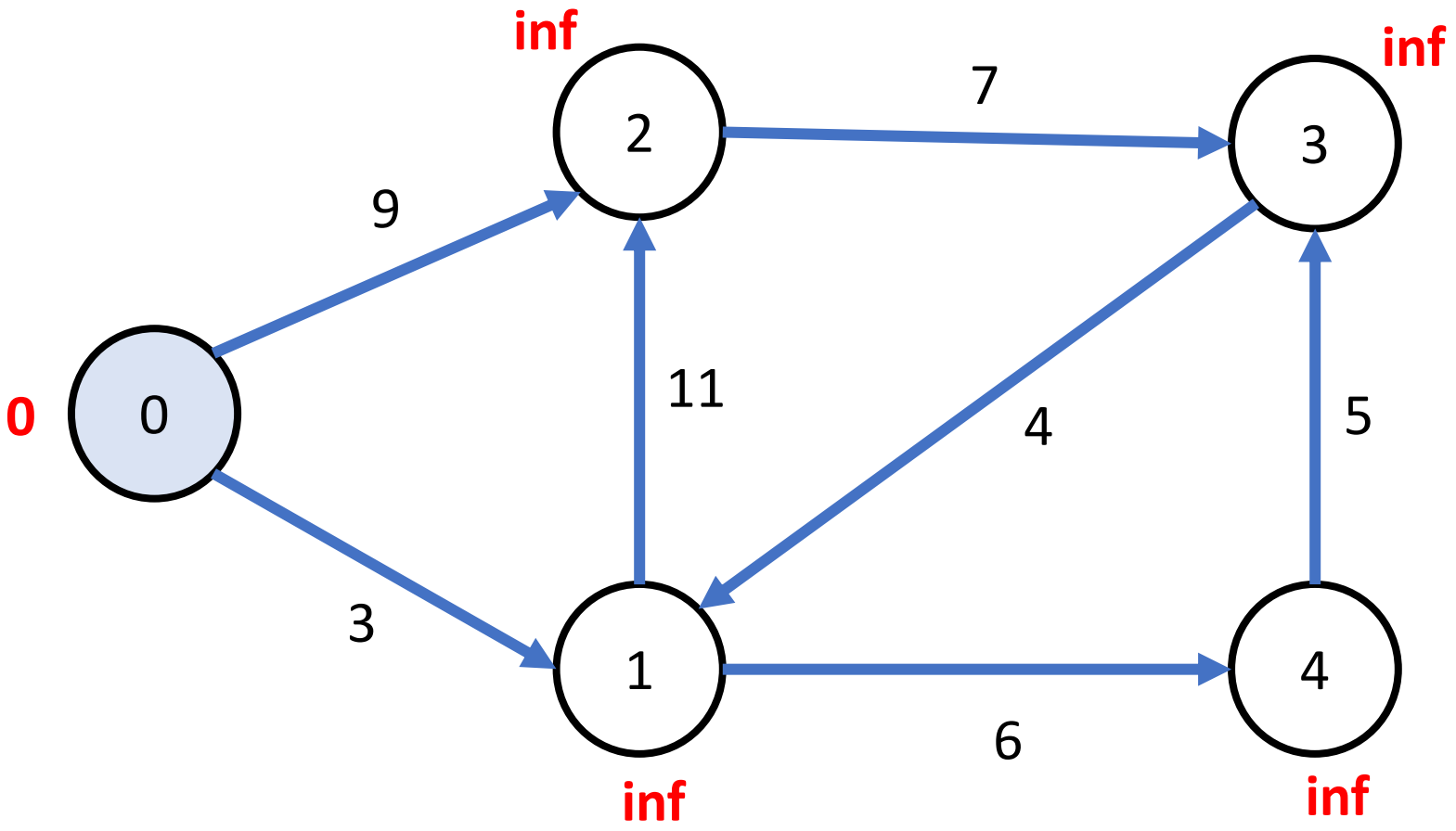
$s = 0$



El algoritmo de Dijkstra

Ejemplo

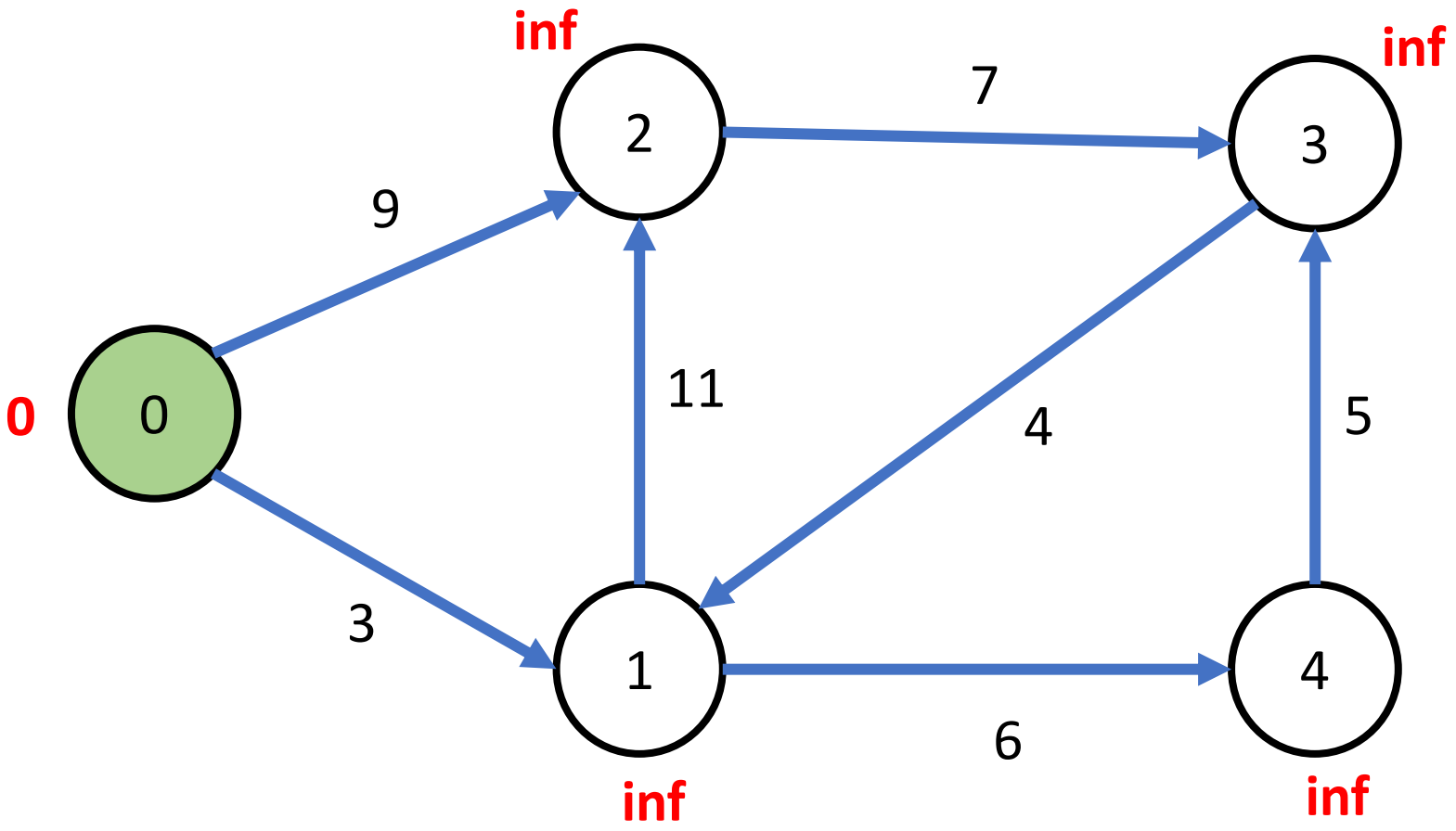
$s = 0$



El algoritmo de Dijkstra

Ejemplo

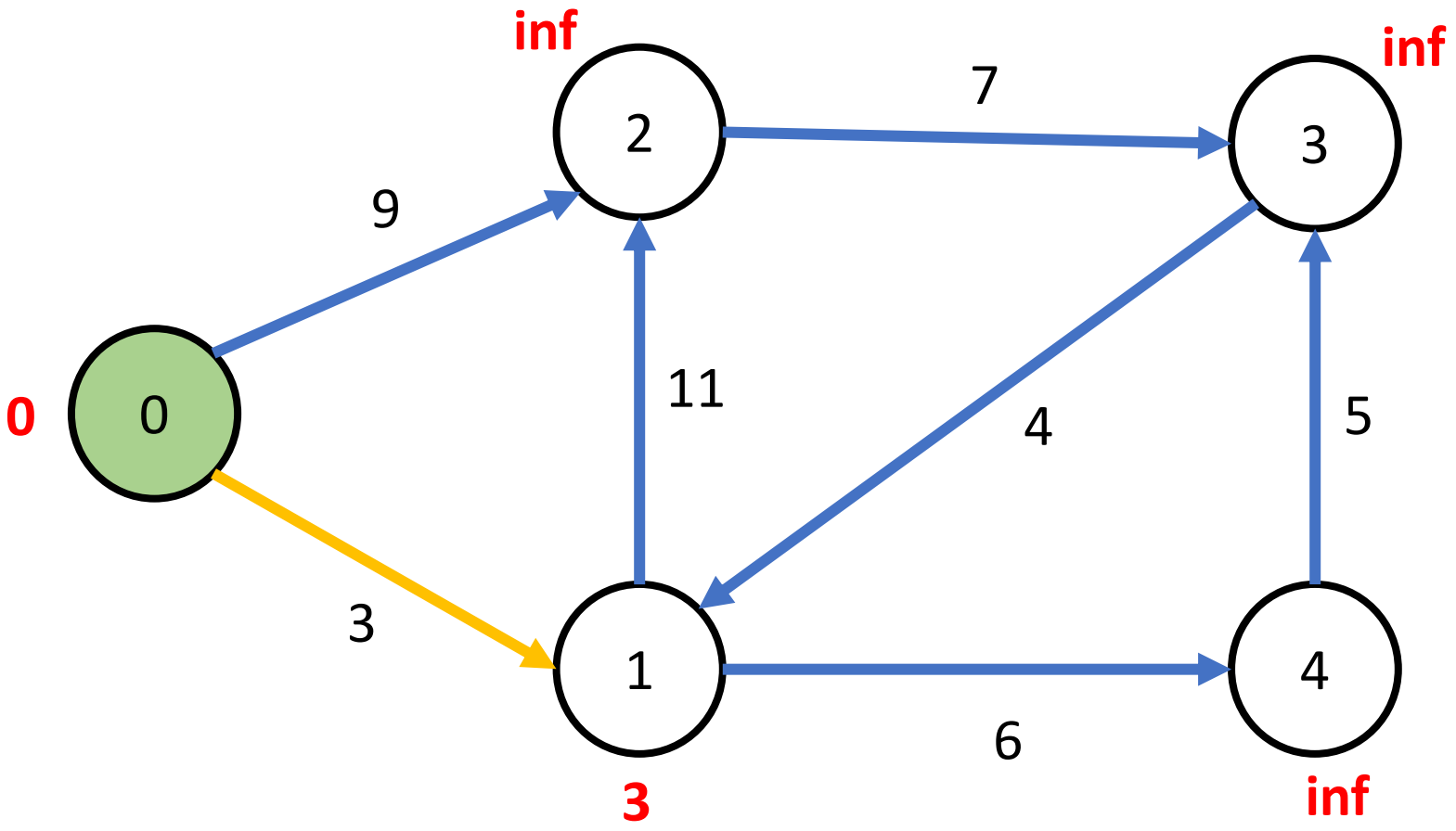
$s = 0$



El algoritmo de Dijkstra

Ejemplo

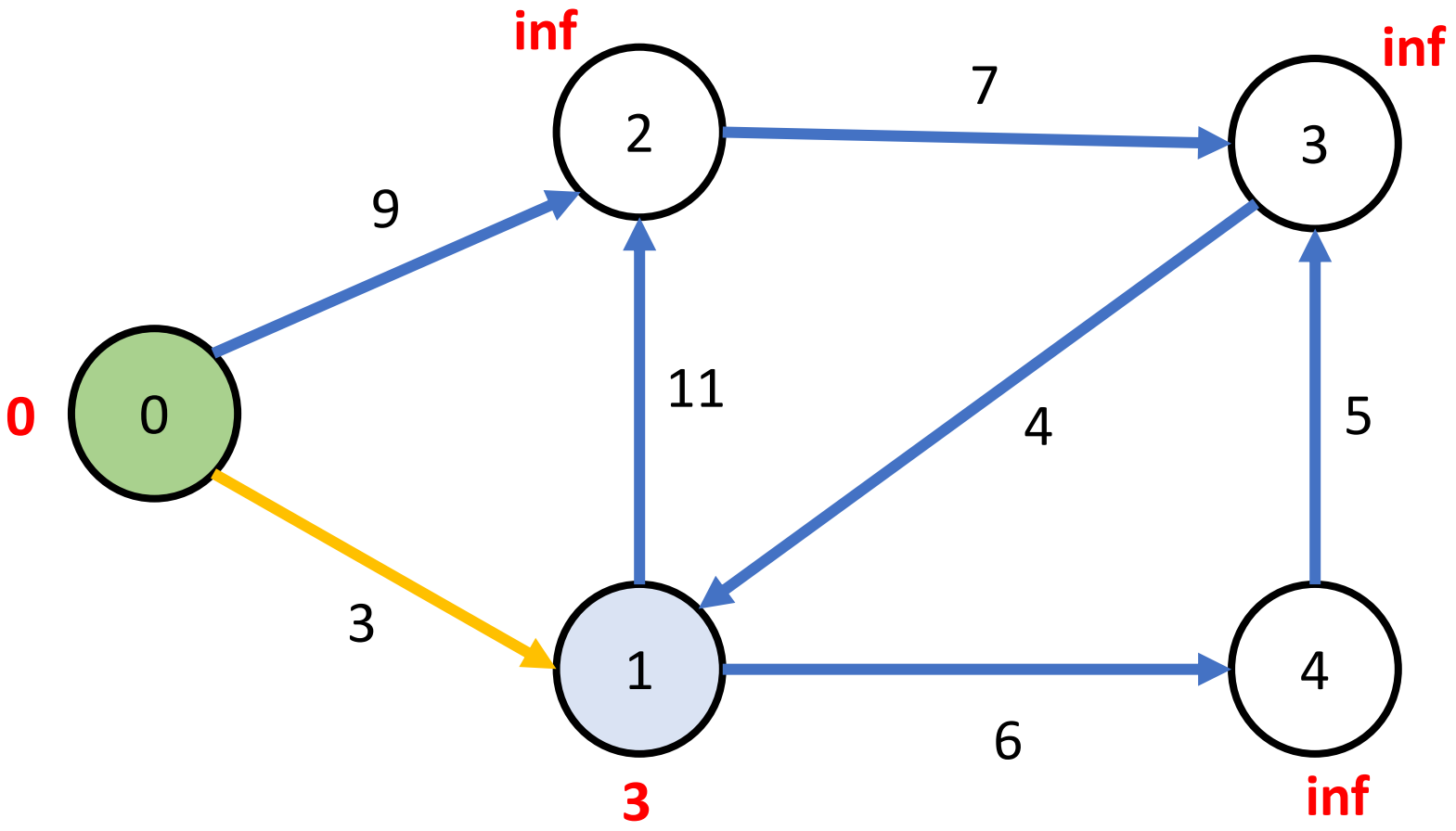
$s = 0$



El algoritmo de Dijkstra

Ejemplo

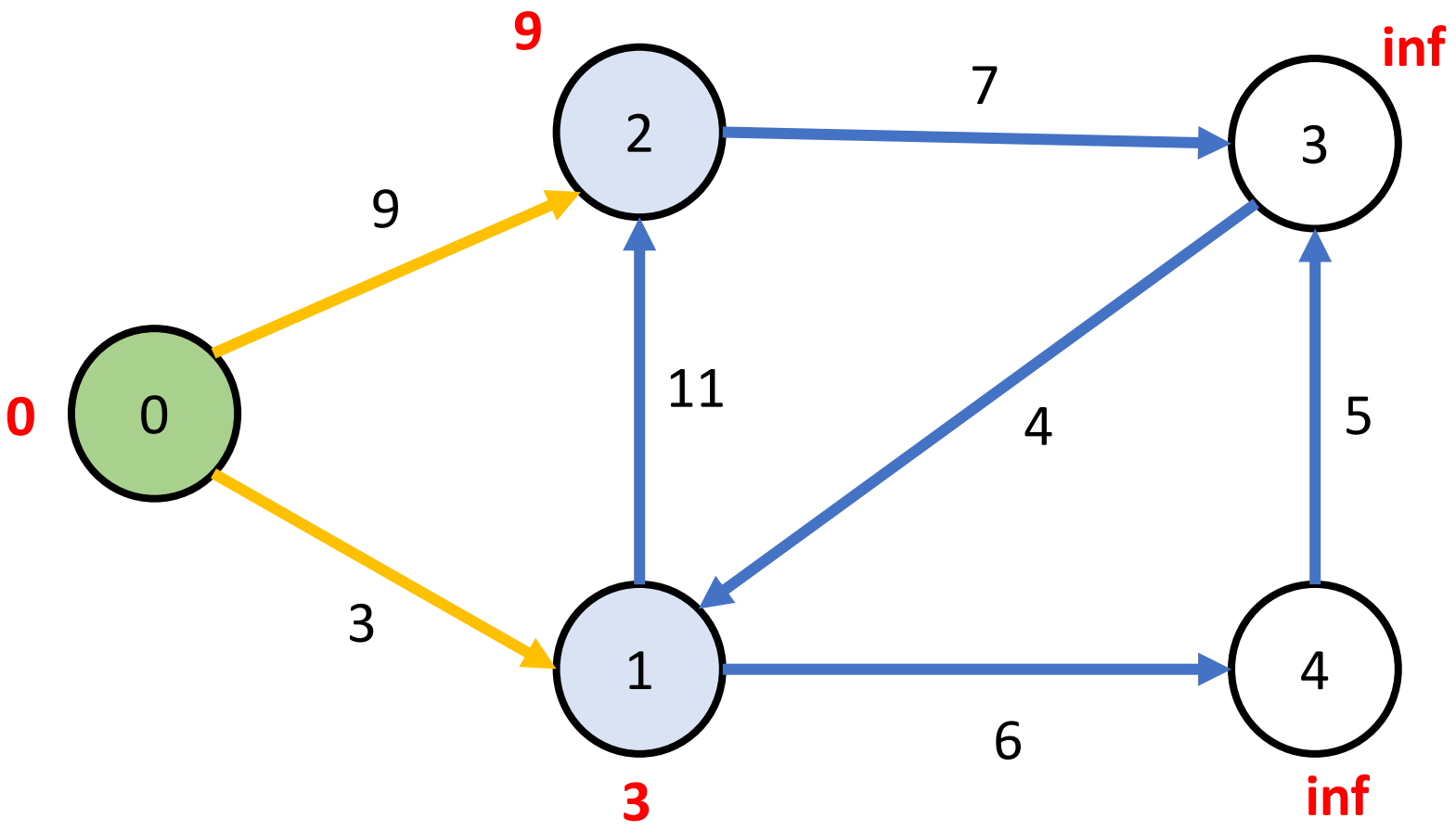
$s = 0$



El algoritmo de Dijkstra

Ejemplo

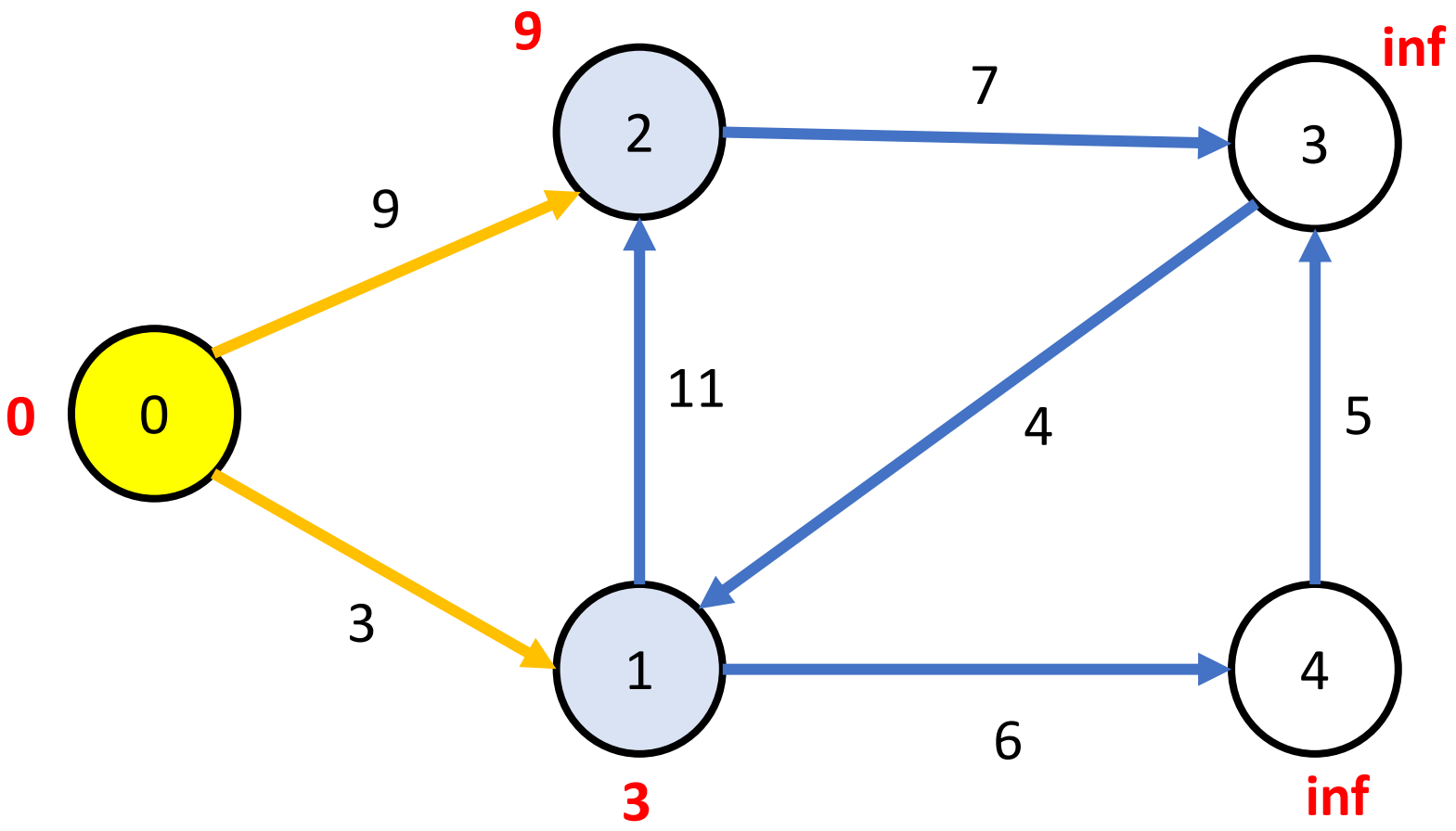
$s = 0$



El algoritmo de Dijkstra

Ejemplo

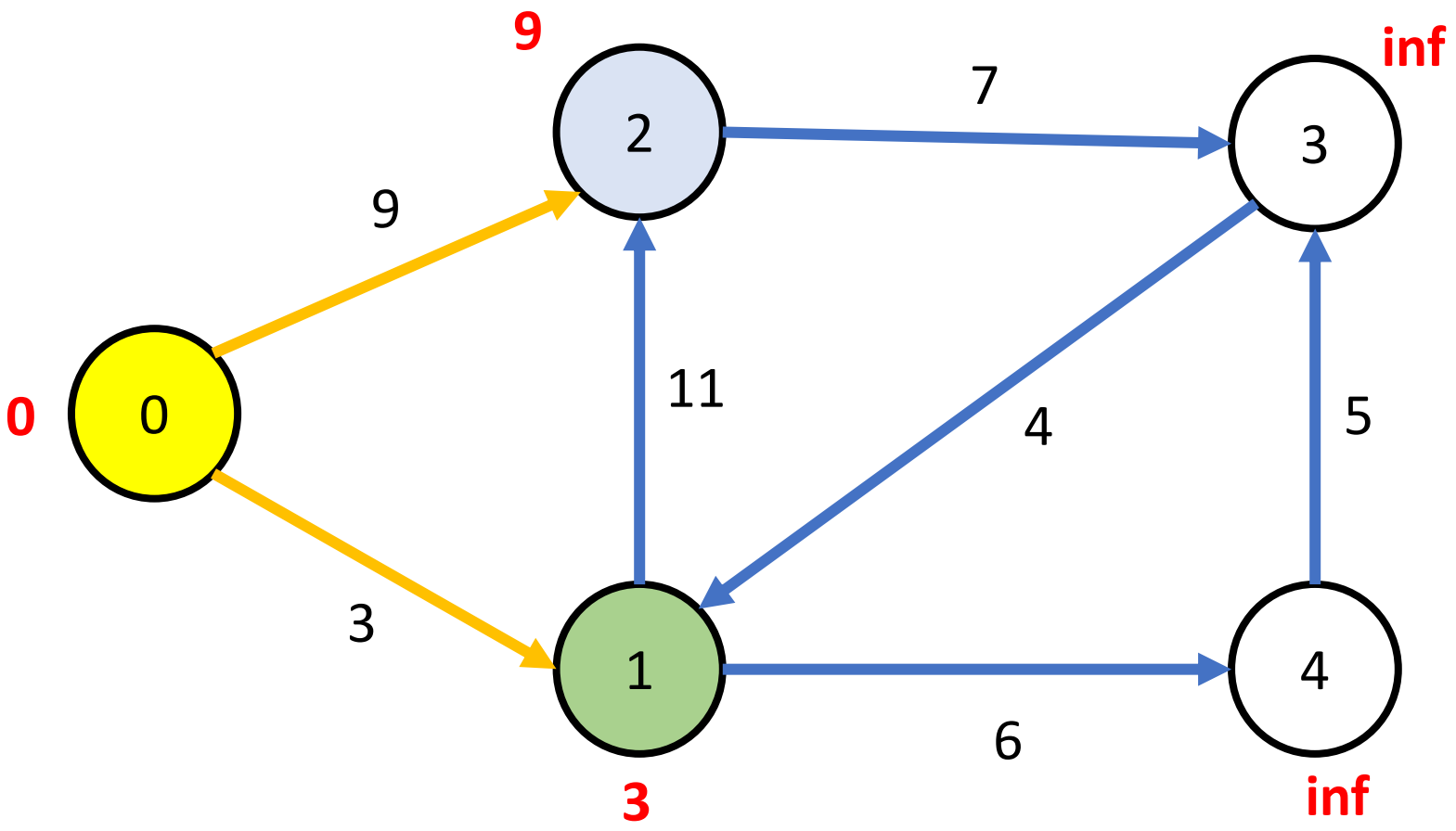
$s = 0$



El algoritmo de Dijkstra

Ejemplo

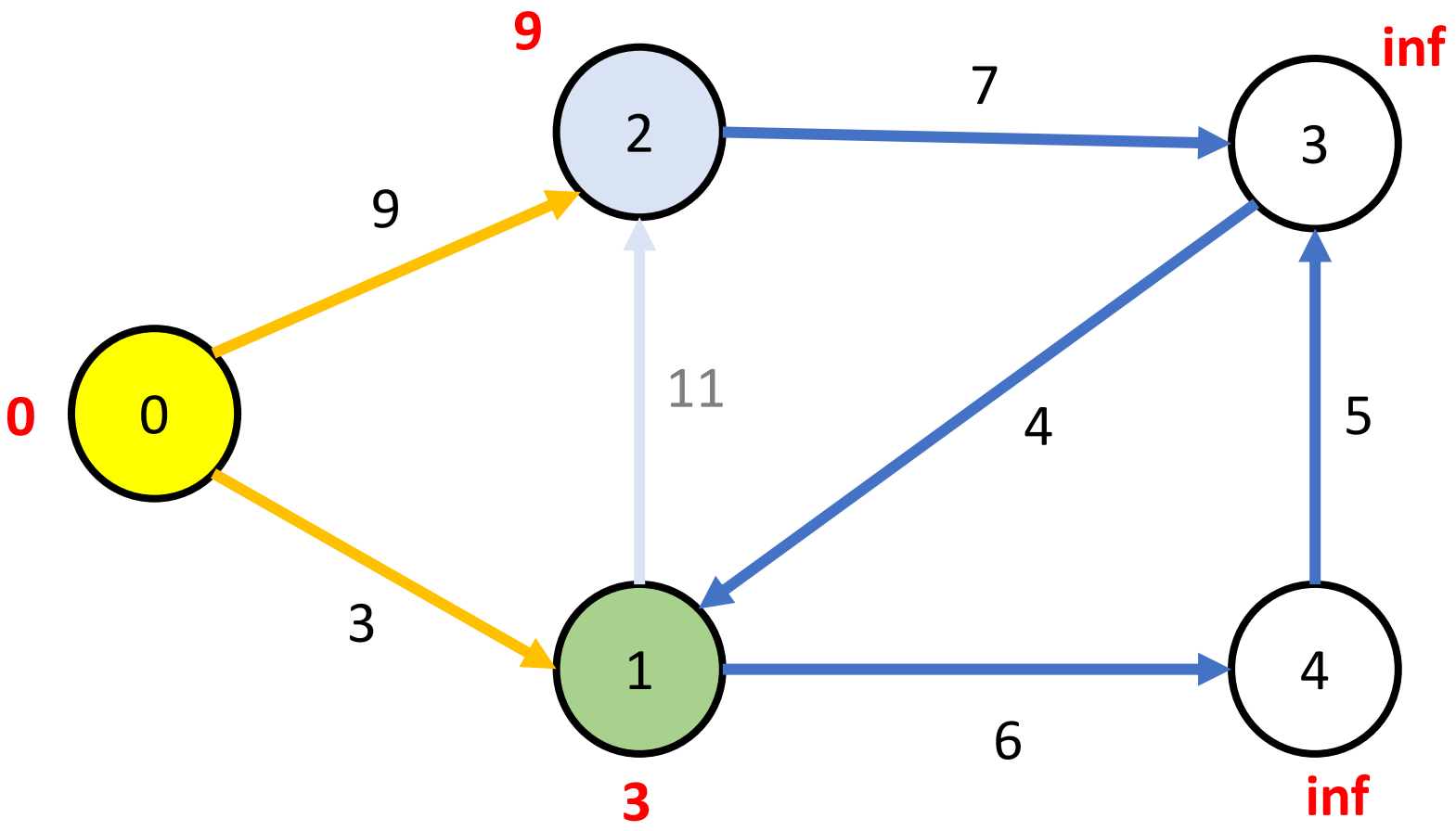
$s = 0$



El algoritmo de Dijkstra

Ejemplo

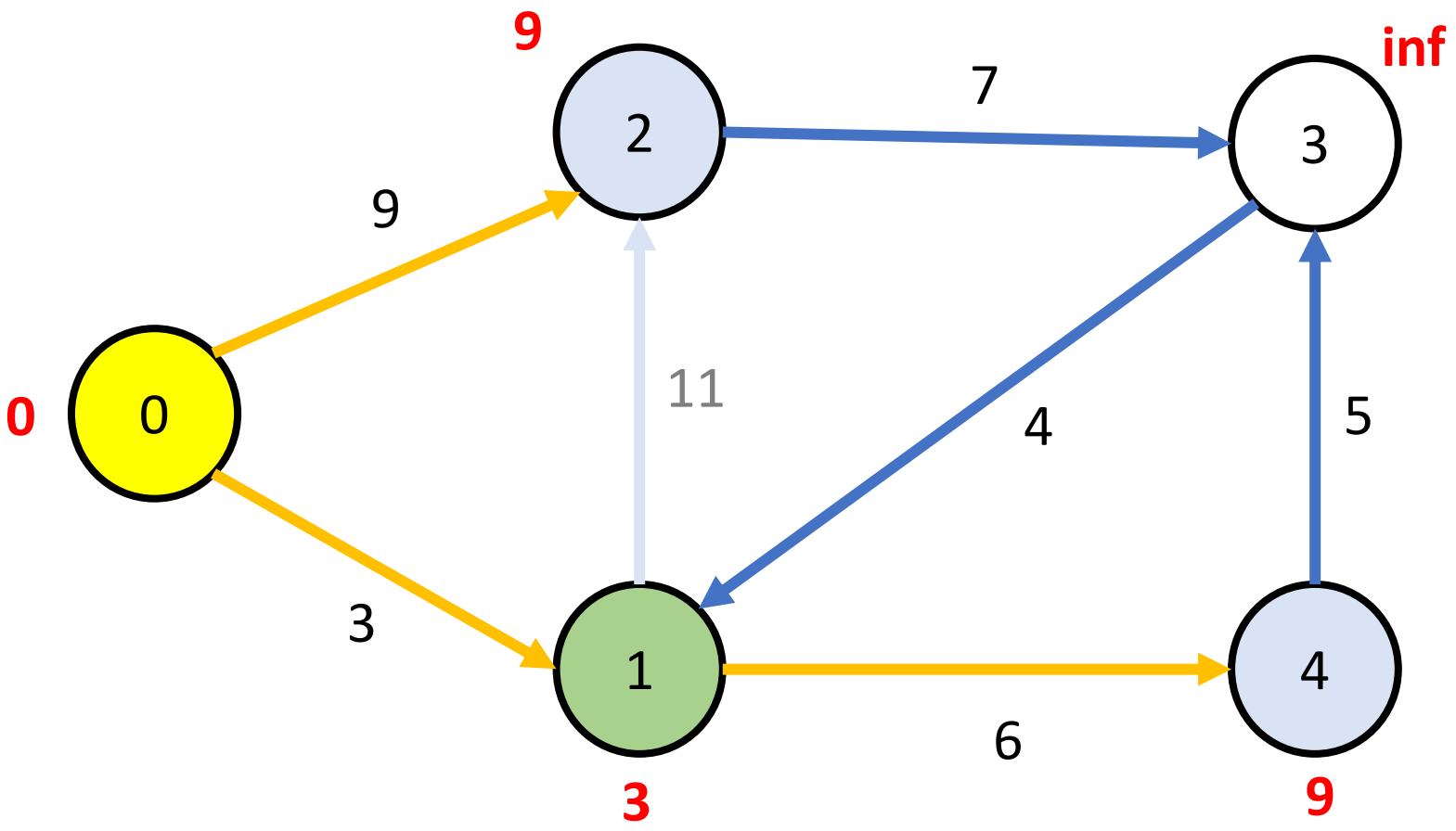
$s = 0$



El algoritmo de Dijkstra

Ejemplo

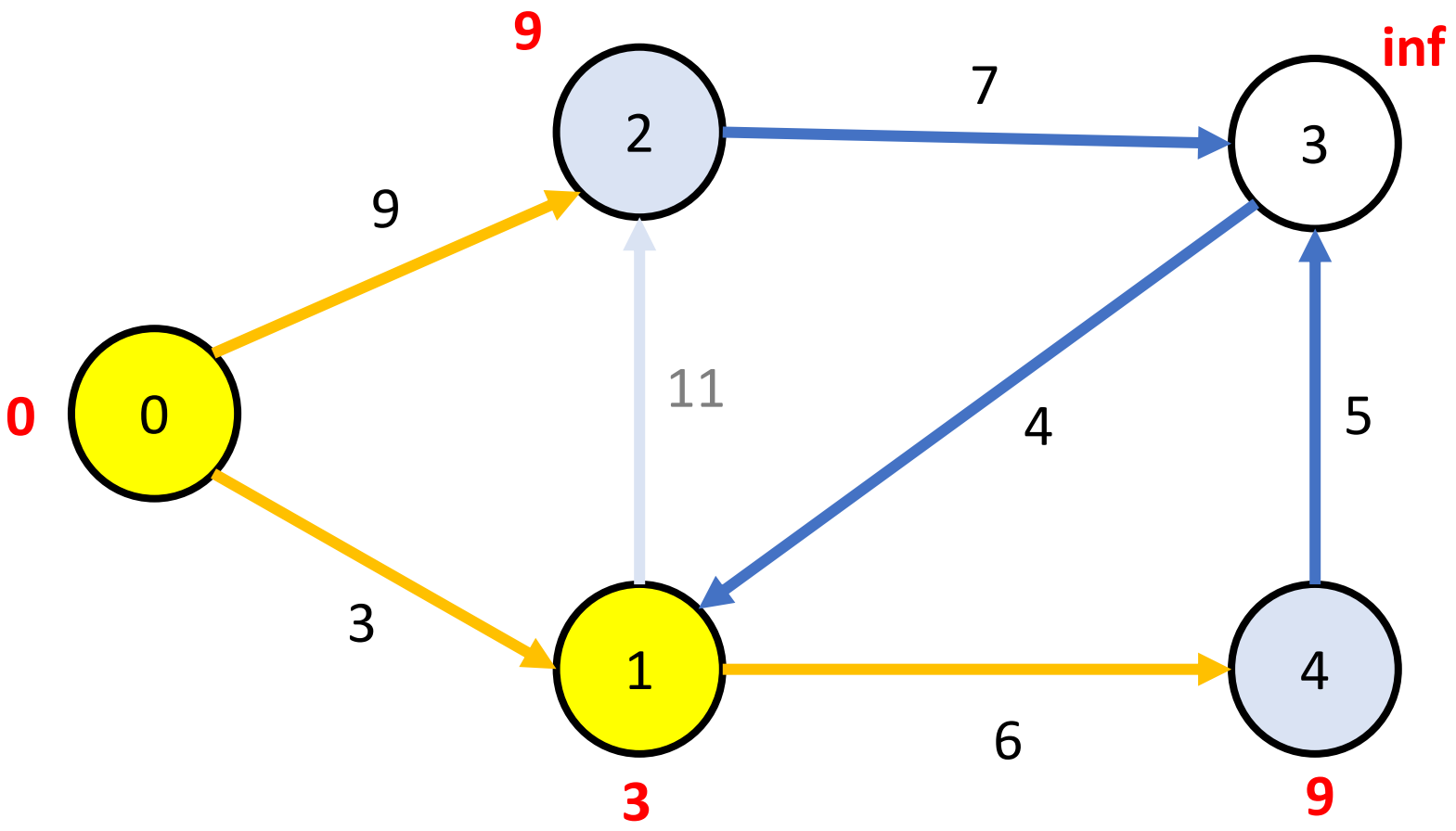
$s = 0$



El algoritmo de Dijkstra

Ejemplo

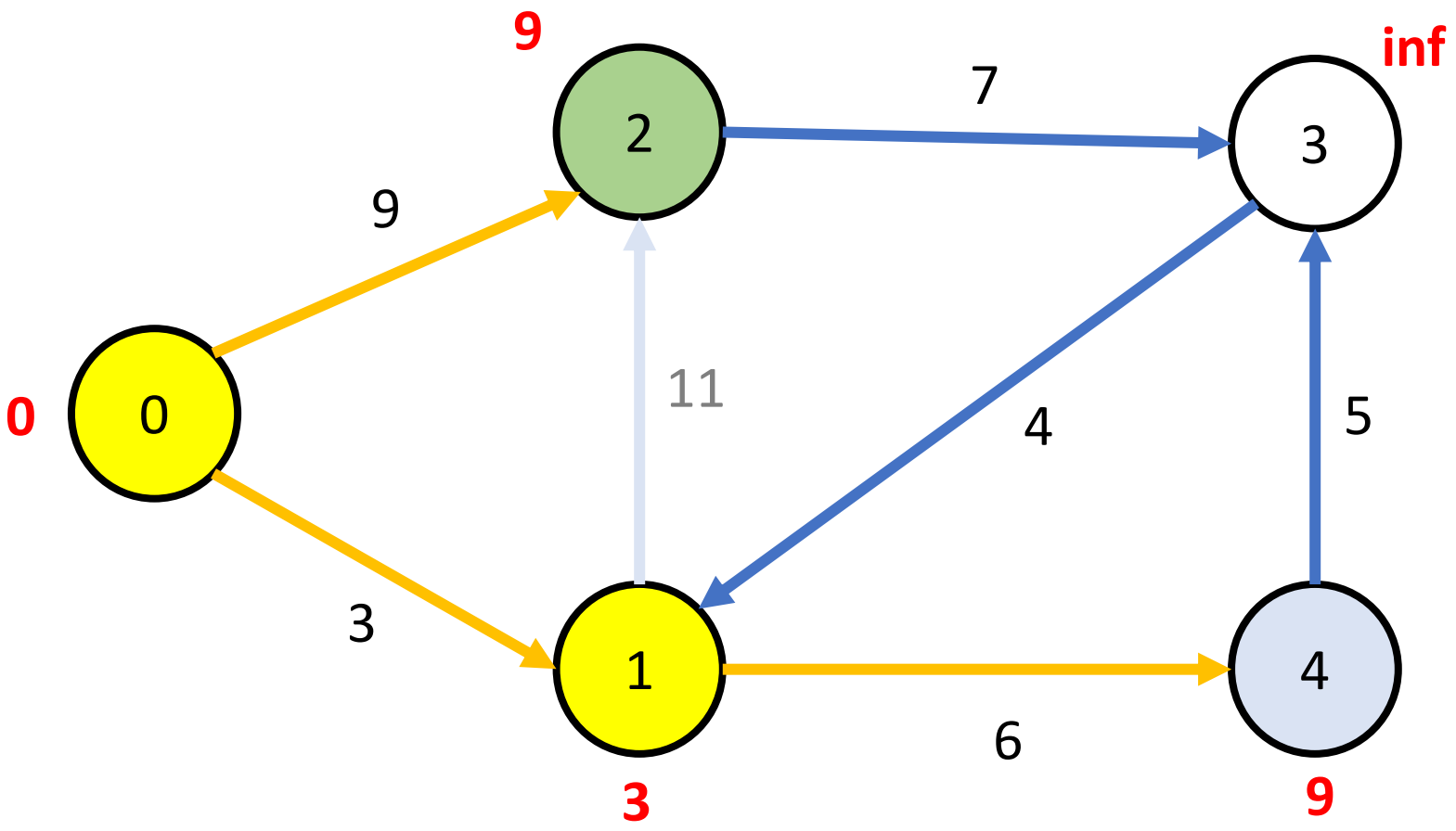
$s = 0$



El algoritmo de Dijkstra

Ejemplo

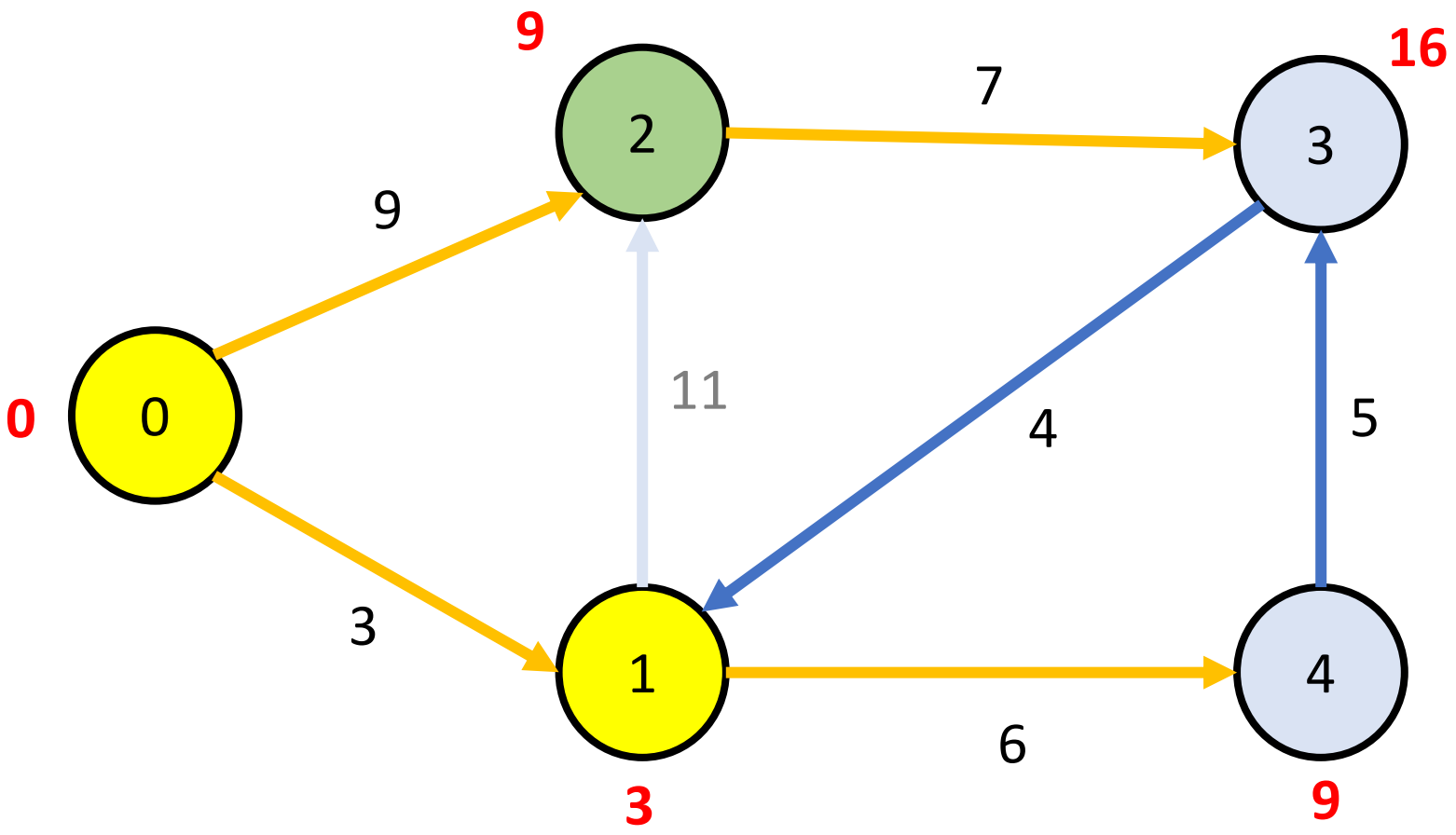
$s = 0$



El algoritmo de Dijkstra

Ejemplo

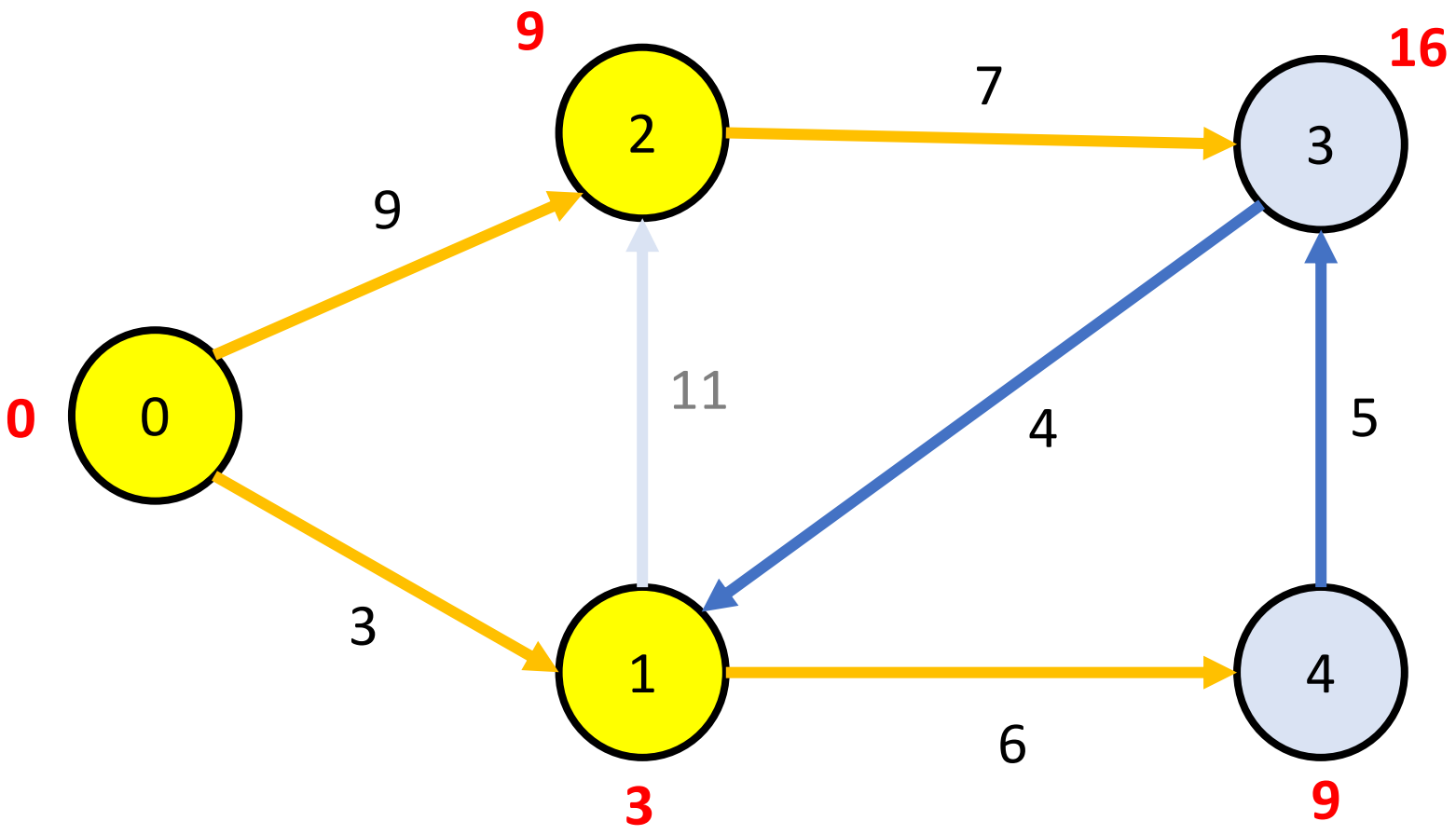
$s = 0$



El algoritmo de Dijkstra

Ejemplo

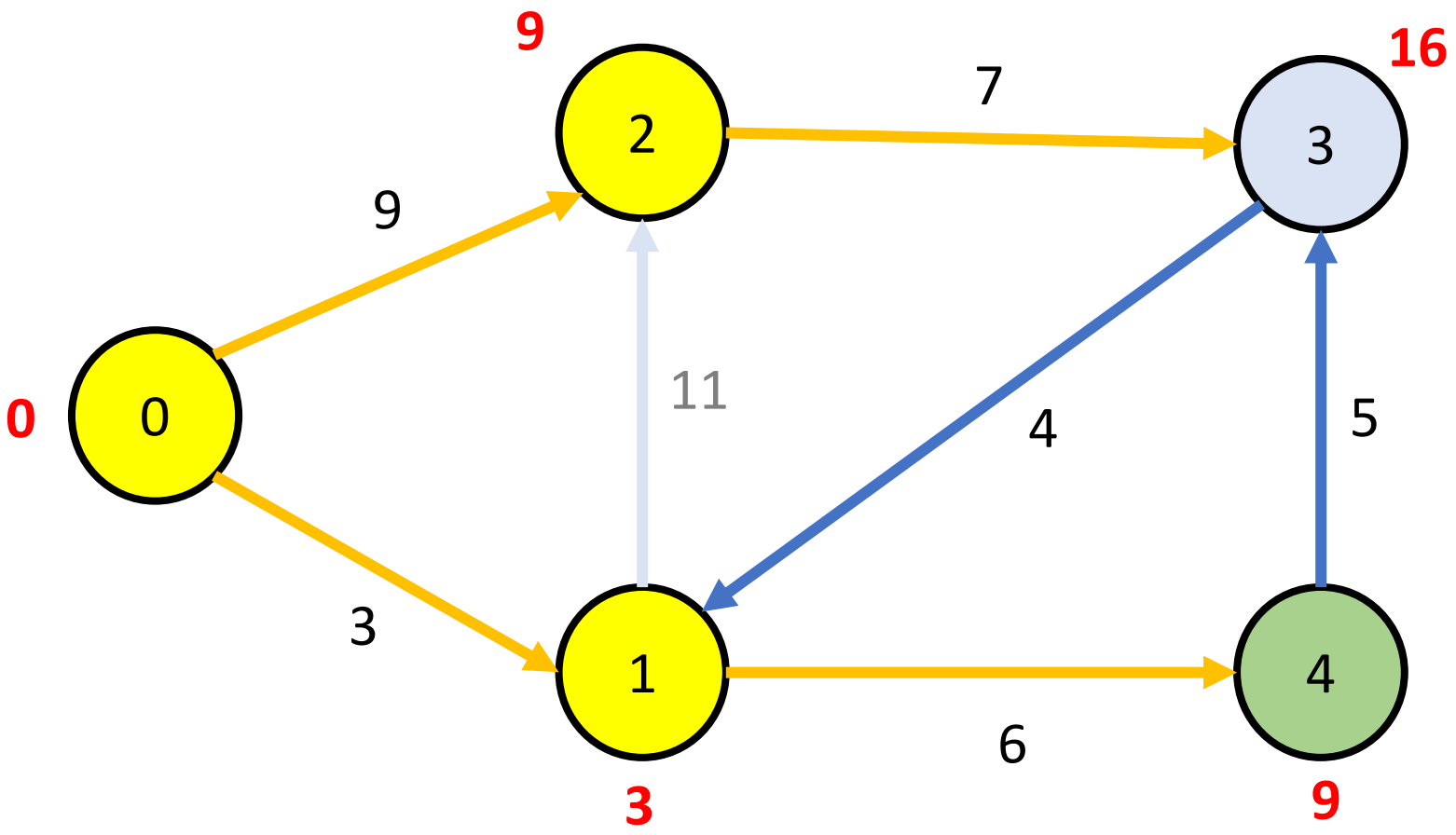
$s = 0$



El algoritmo de Dijkstra

Ejemplo

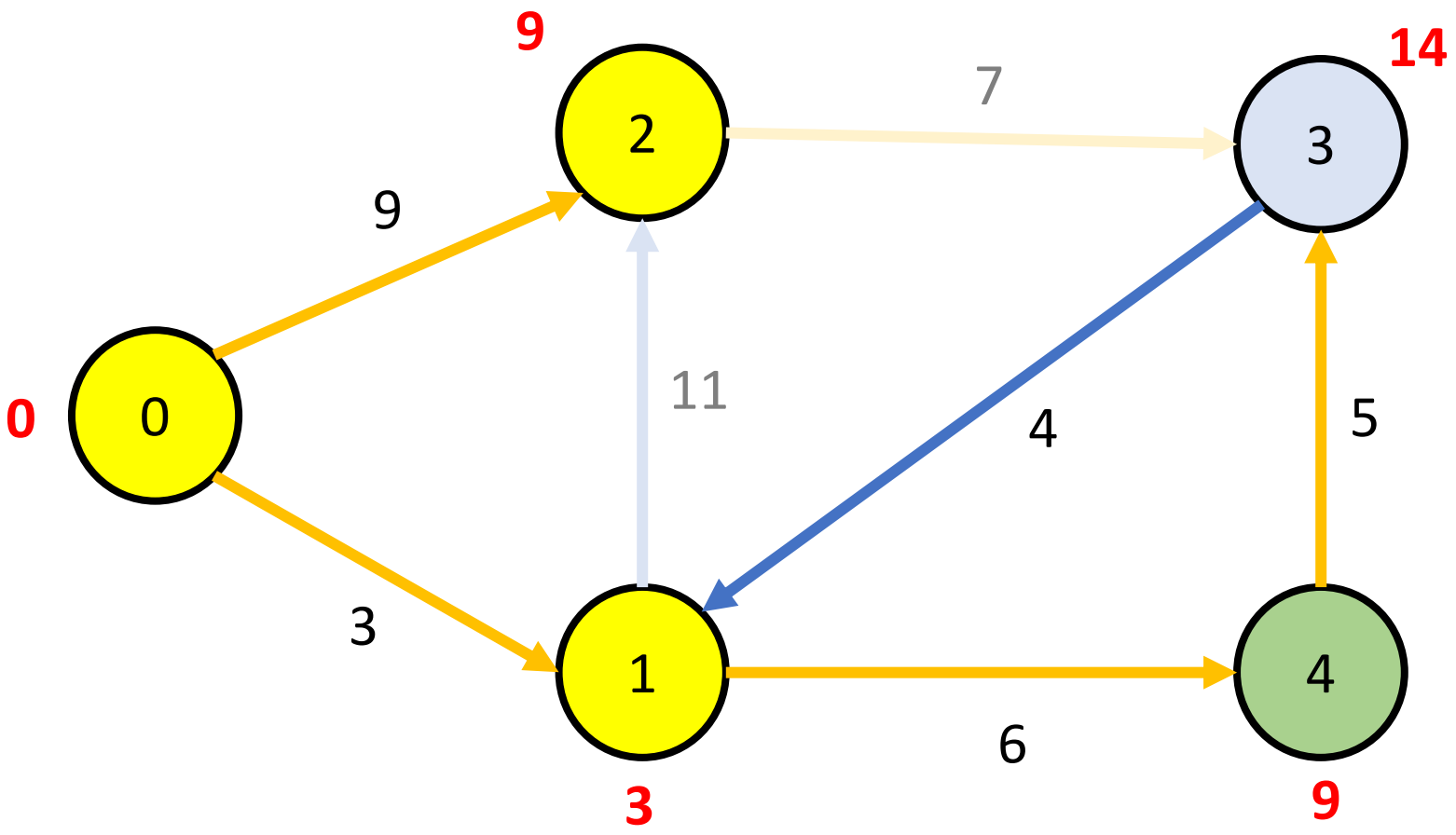
$s = 0$



El algoritmo de Dijkstra

Ejemplo

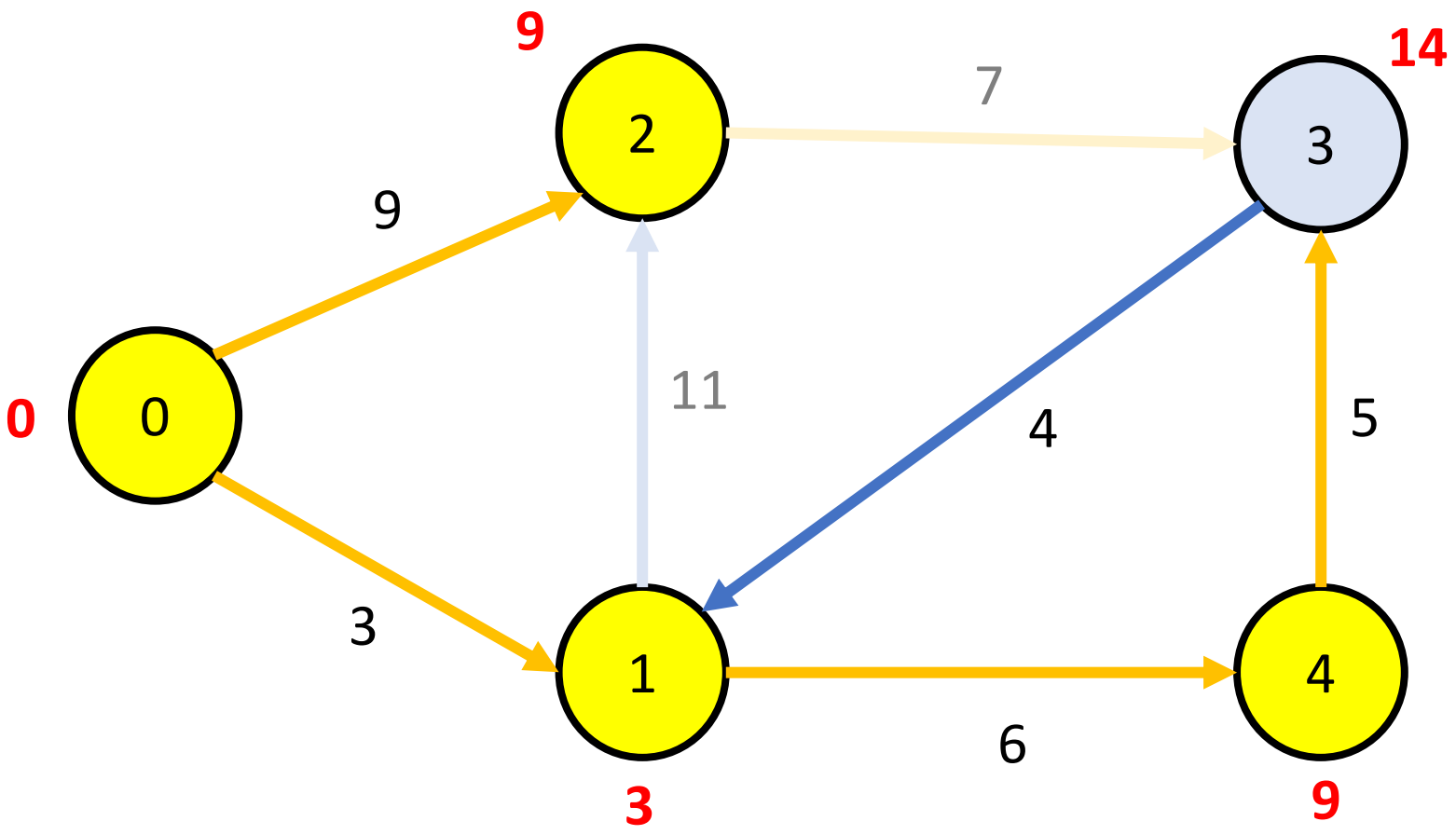
$s = 0$



El algoritmo de Dijkstra

Ejemplo

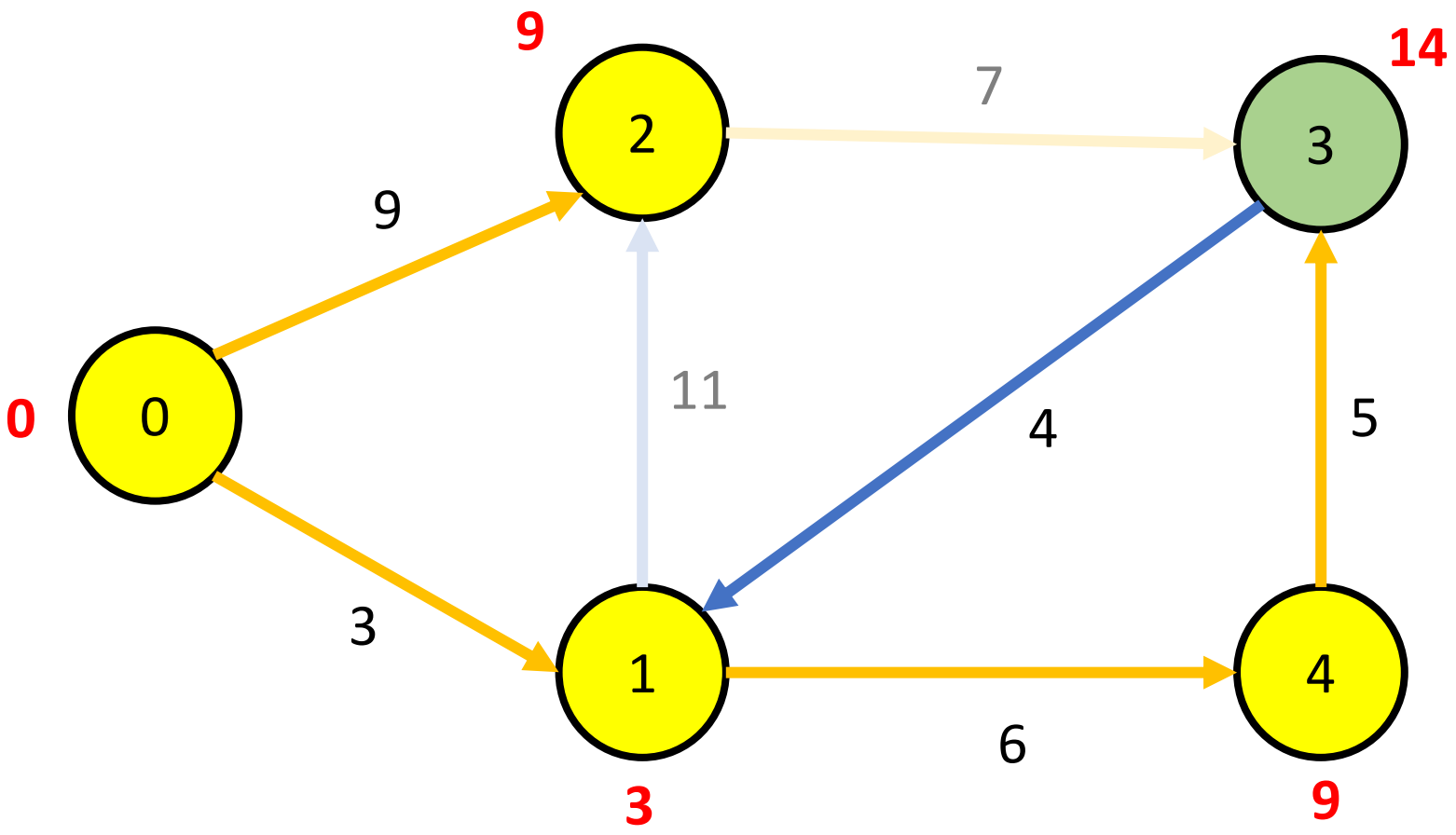
$s = 0$



El algoritmo de Dijkstra

Ejemplo

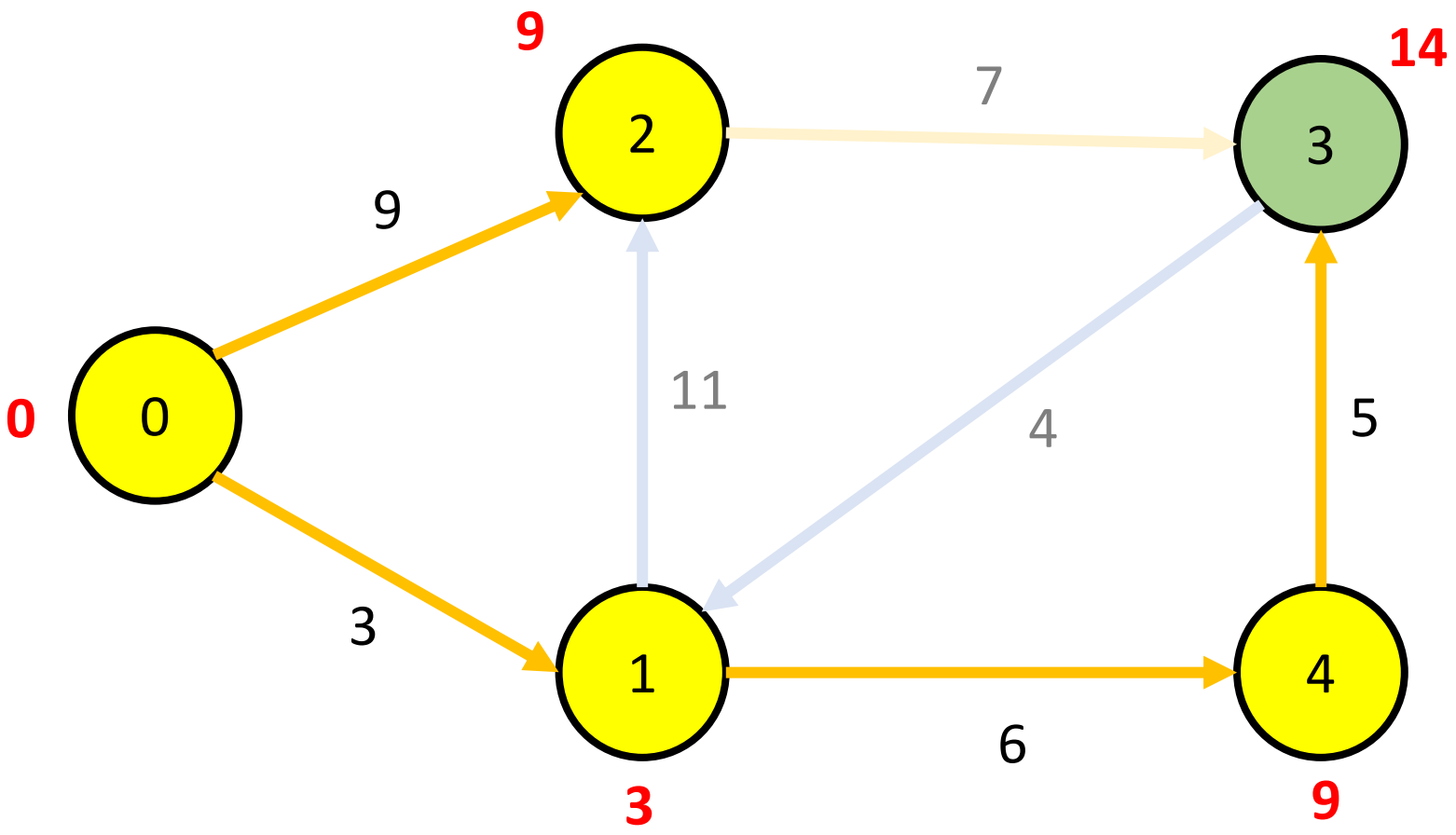
$s = 0$



El algoritmo de Dijkstra

Ejemplo

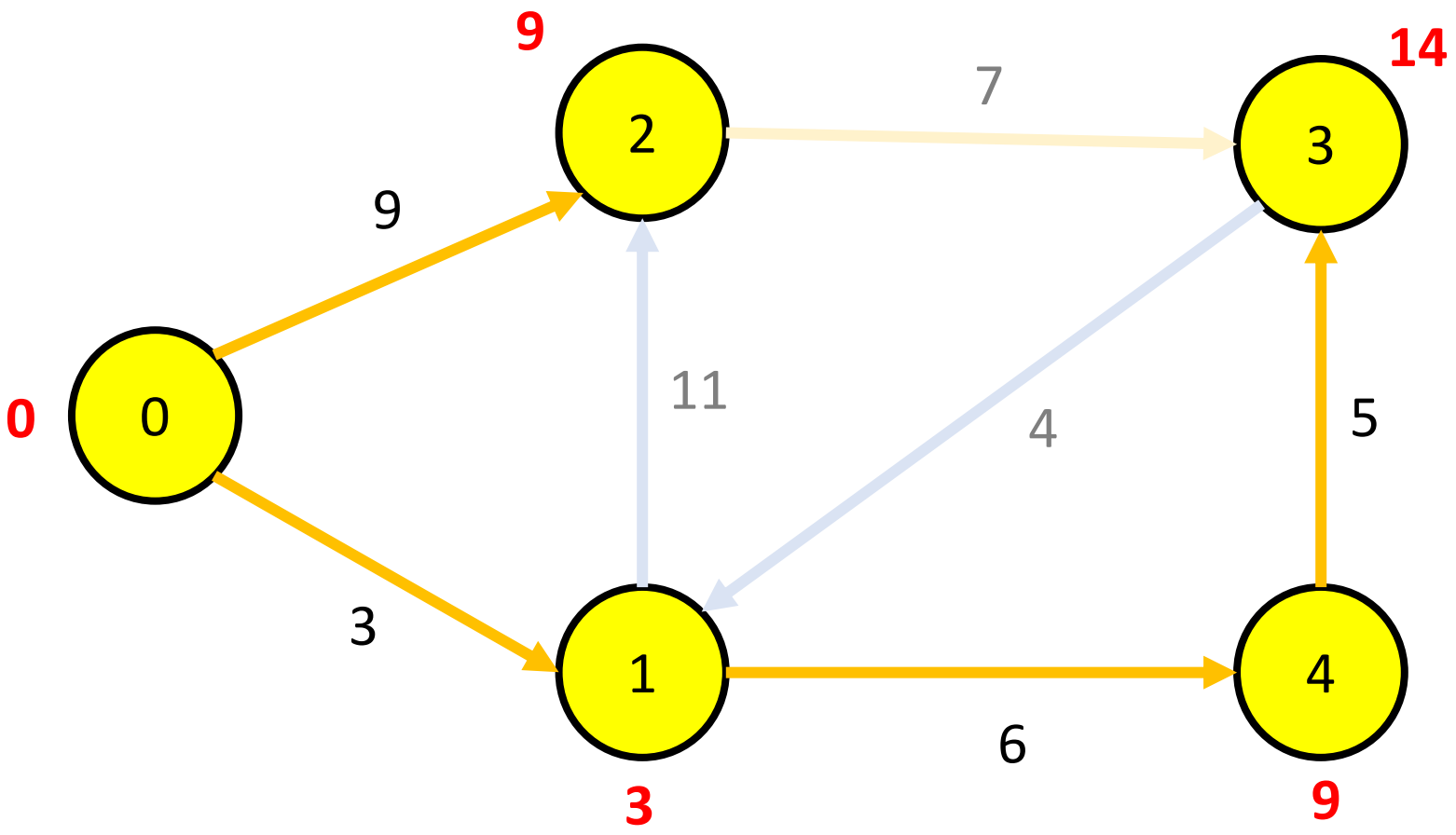
$s = 0$



El algoritmo de Dijkstra

Ejemplo

$s = 0$



El algoritmo de Dijkstra

Complejidad

Tenemos N nodos y E aristas.

Complejidad: $O(N^2 + E)$.

Si usamos un set o priority_queue para obtener rápidamente el nodo pendiente de menor costo, la complejidad mejora a $\boldsymbol{O(E\ log\ E)}$.

El algoritmo de Dijkstra

Código

```cpp
using ll = long long;
const ll INF = 4e18;

vector<ll> dijkstra(vector<vector<pair<ll,ll>>> G, int n, int s){
    // G -> Vector de vector de pares, guarda el grafo con pesos.
    // n -> Cantidad de nodos.
    // s -> Nodo inicial
    vector<ll> dis(n+1, INF);
    dis[s] = 0;
    // Vamos a guardar pares {w, nodo} donde w es el costo con el que llegamos al nodo.
    // Es importante que primero este el costo w, ya que de ese modo se ordenara de acuerdo a los pesos.
    priority_queue<pair<ll,ll>, vector<pair<ll,ll>>, greater<pair<ll,ll>>> pq;  // Cola de prioridad que nos da los menores primero.
    pq.push({0, s});    // El primer nodo pendiente es s, con peso 0.
    while(!pq.empty()){     // Mientras existan nodos pendientes.
        pair<ll,ll> menor = pq.top();     // Recuperamos el nodo con menor distancia.
        pq.pop();                         // Lo quitamos de pendientes.
        int w = menor.first;              // Costo del nodo actual
        int nodo = menor.second;          // Nodo actual
        if(dis[nodo] < w) continue;       // Anteriormente encontramos una distancia menor a la del costo actual, asi que ahora no hacemos nada
        for(auto to : G[nodo]){
            int v = to.first;             // Nodo vecino
            int cost = to.second;         // Costo para ir al nodo v
            if(dis[nodo] + cost < dis[v]){      // Se puede mejorar la distancia minima al nodo v
                dis[v] = dis[nodo] + cost;      // Actualizamos la distancia mejorada
                pq.push({dis[v], v});           // Hicimos una mejora, asi que lo insertamos en pendientes al nodo mejorado
            }
        }
    }
    return dis;
}
```

El algoritmo de Floyd-Warshall

Definición

Dado un grafo con pesos **cualquiera** (positivos o negativos):

- El algoritmo de Floyd-Warshall nos permite calcular el camino más corto **para cada par de nodos**.

El algoritmo de Floyd-Warshall

Algoritmo

- Sea $d[u][v]$ la distancia del nodo u al nodo v.
- Inicialmente todas las distancias son infinitas. $d[u][v] = \infty$
- La distancia de un nodo al mismo nodo es 0: $d[u][u] = 0$
- Revisamos la aristas del grafo. Por cada arista del nodo u al nodo v con costo w, actualizamos la distancia: $d[u][v] = \min(d[u][v], w)$
- Para cada nodo k, que servirá de “intermediario” entre cada par de nodos:
  - Para cada par de nodos u, v:
    - $d[u][v] = \min(d[u][v], d[u][k] + d[k][v])$

El algoritmo de Floyd-Warshall

Complejidad

Tenemos N nodos y E aristas.

- Vamos por cada arista para actualizar distancias iniciales. $O(E)$
- Vamos por cada nodo intermediario k. $O(N)$
  - Vamos por cada nodo u. $O(N)$
    - Vamos por cada nodo v. $O(N)$

Complejidad: $\boldsymbol{O(N^3 + E)}$.

El algoritmo de Floyd-Warshall

Código

```cpp
vector<vector<ll>> floydWarshall(vector<vector<pair<ll,ll>>> G, int n){
    vector<vector<ll>> dis(n+1, vector<ll>(n+1, INF));
    for(int i=1; i<=n; ++i){
        dis[i][i] = 0;
    }
    for(int u=1; u<=n; ++u){    // Revisamos todas las aristas del grafo.
        for(auto to : G[u]){
            ll v = to.first, w = to.second;
            dis[u][v] = min(dis[u][v], w);
            dis[v][u] = min(dis[v][u], w);  // Si el grafo es bidireccional
        }
    }
    for(int k=1; k<=n; ++k){        // Intermediario
        for(int u=1; u<=n; ++u){       // Nodo u
            for(int v=1; v<=n; ++v){    // Nodo v
                dis[u][v] = min(dis[u][v], dis[u][k] + dis[k][v]);
            }
        }
    }
    return dis;
}
```

Union-Find

Definición

Union-Find es una estructura de datos que nos permite manejar distintos conjuntos pudiendo hacer las siguientes operaciones:

- $Find(x)$: Devuelve el ID del conjunto al que pertenece el elemento x.
- $Join(x, y)$: Une los conjuntos a los que pertenecen x, y en un solo conjunto.

Union-Find

Código

```
struct unionFind {

    vector<int> p;              // Guarda el ID del conjunto al que pertenece cada elemento.
    unionFind(int n) : p(n) {
        for(int i=0; i<n; ++i){
            p[i] = i;           // Inicialmente el id de cada elemento es él mismo.
        }
    }

    int find(int x){
        return p[x] = find(p[x]);
    }

    void join(int x, int y){
        x = find(x);            // Buscamos la ID del conjunto al que pertenece x.
        y = find(y);           // Buscamos la ID del conjunto al que pertenece y.
        p[y] = x;            // Ahora ambos conjuntos tienen el ID del conjunto x. Se unieron.
    }

};
```

Union-Find

Código + Ejemplo

```cpp
struct unionFind {
    vector<int> p;
    unionFind(int n) : p(n, -1) {}
    int find(int x){
        if(p[x] == -1) return x;
        return p[x] = find(p[x]);
    }
    bool join(int x, int y){
        x = find(x), y = find(y);
        if(x == y) return 0;
        p[y] = x;
        return 1;
    }
};

int main(){
    int n = 5;
    unionFind conjuntos(n+1);
    conjuntos.join(5, 3);
    conjuntos.join(1, 3);
    for(int i=1; i<=n; ++i){
        cout << "ID de " << i << ": " << conjuntos.find(i) << endl;
    }
}
```

Salida:

Received Output: Copy

```
ID de 1: 1
ID de 2: 2
ID de 3: 1
ID de 4: 4
ID de 5: 1
```

Union-Find

Complejidad

$Find(x)$: $O(\log n)$
$Join(x, y)$: $O(\log n)$

Ambas operaciones de Union-Find tienen una complejidad de $O(\log n)$

Árbol de expansión mínima

Definición

Un árbol de expansión mínima (Minimum Spanning Tree / MST) es un árbol que **conecta todos los nodos de un grafo**, es decir que existirá un camino entre cualquier par de nodos, y el **peso total** del árbol es lo **mínimo posible**.

Lo podemos encontrar con el algoritmo de Kruskal.

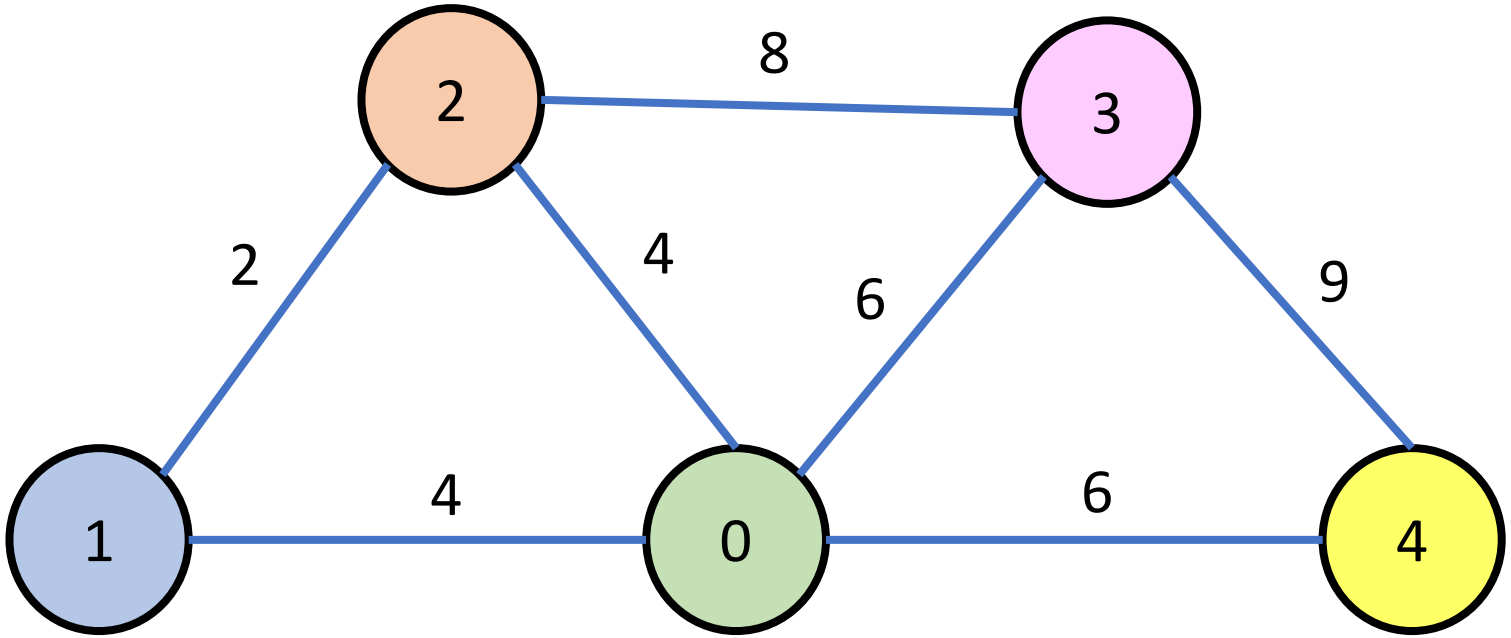
El algoritmo de Kruskal

Algoritmo

- Tenemos una lista de todas las aristas del grafo.
- Ordenamos la lista de aristas de menor peso a mayor peso.
- Recorremos la lista ordenada de aristas.
  - Si la arista actual une dos conjuntos distintos, esta arista forma parte del árbol. Juntamos ambos conjuntos con Union-Find.
  - De otro modo, la arista es innecesaria, no la tomamos en cuenta.
  - Si ya hemos hicimos valer n – 1 aristas, podemos terminar el algoritmo.

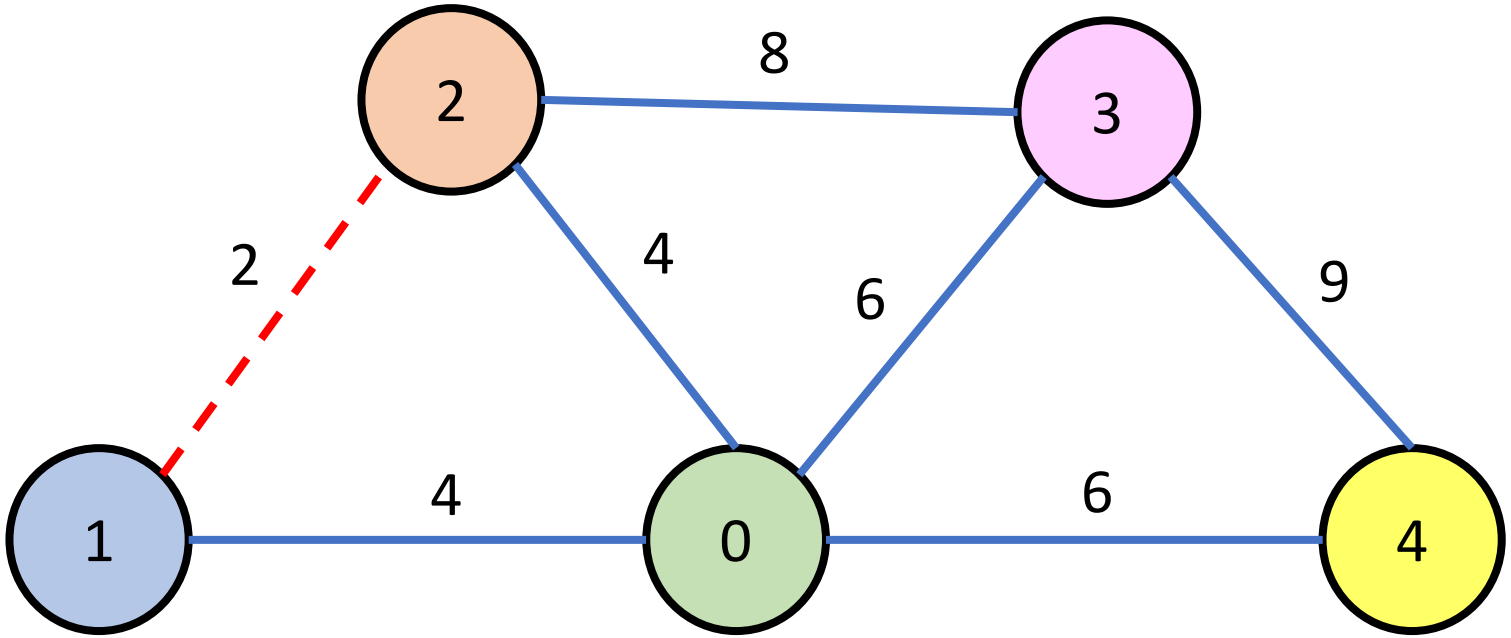
El algoritmo de Kruskal

Ejemplo



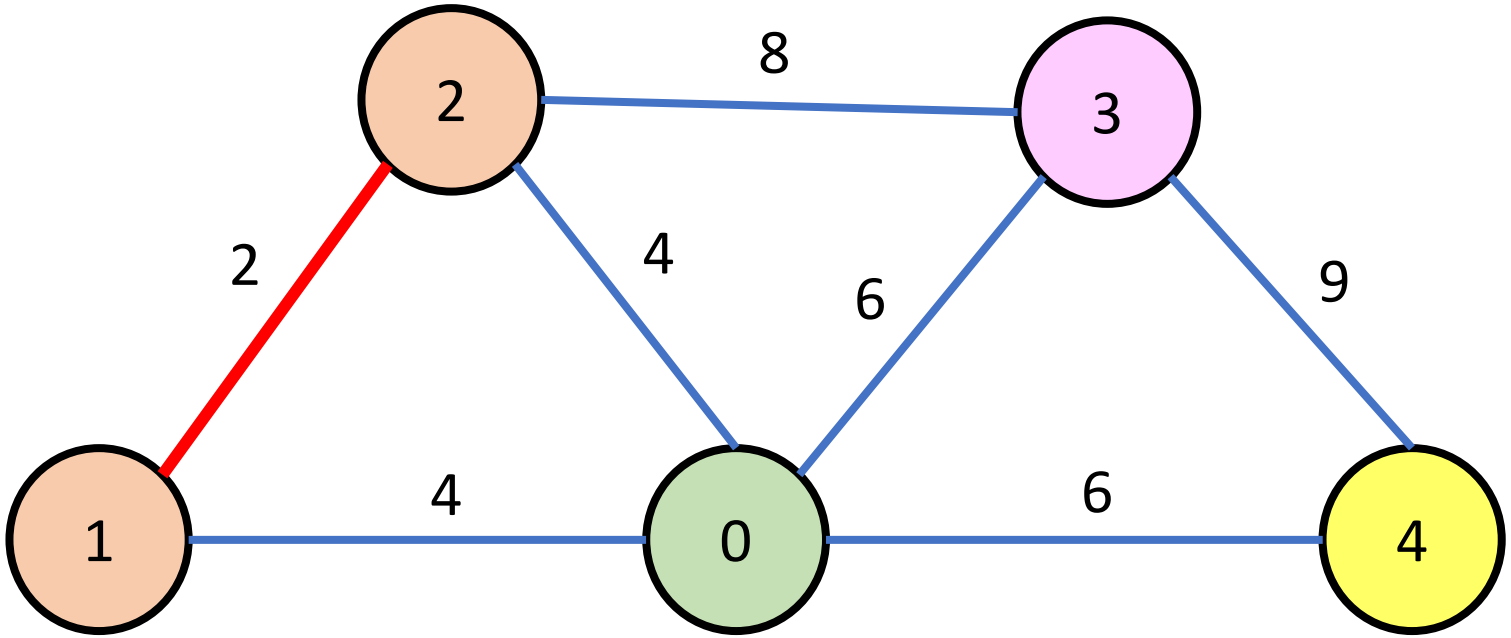
El algoritmo de Kruskal

Ejemplo



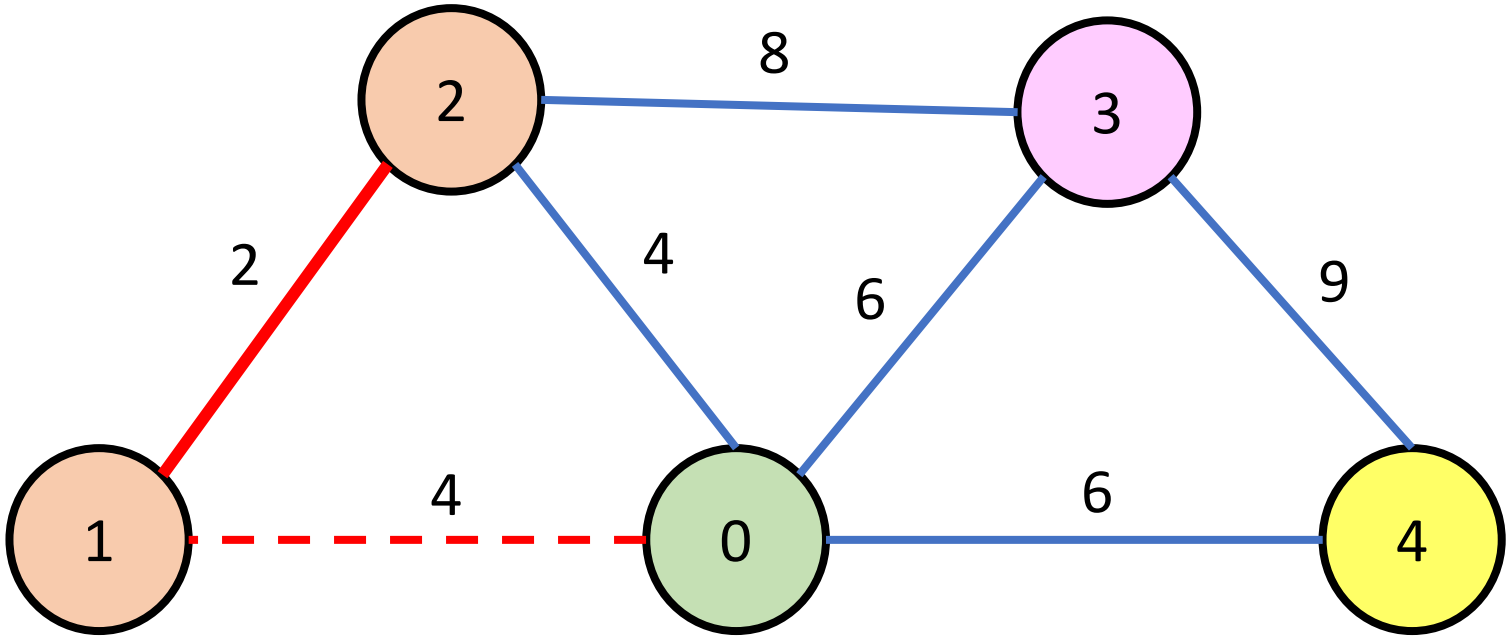
El algoritmo de Kruskal

Ejemplo



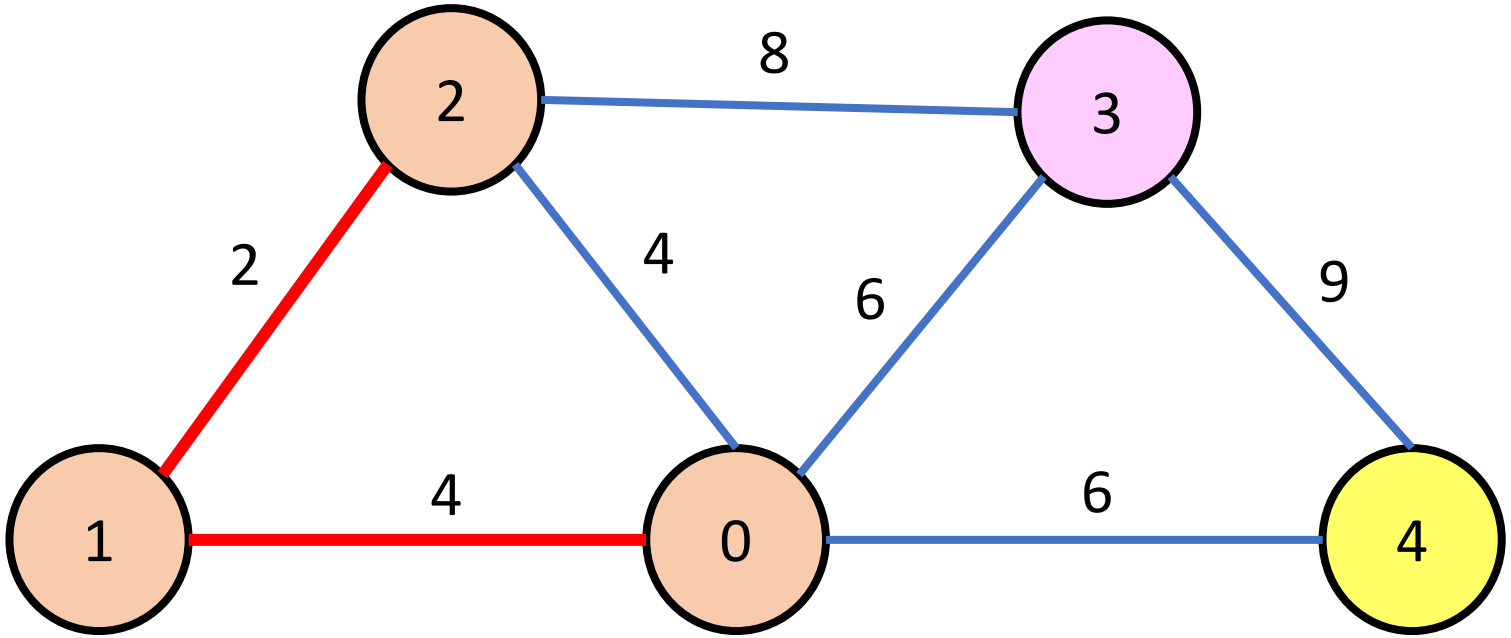
El algoritmo de Kruskal

Ejemplo



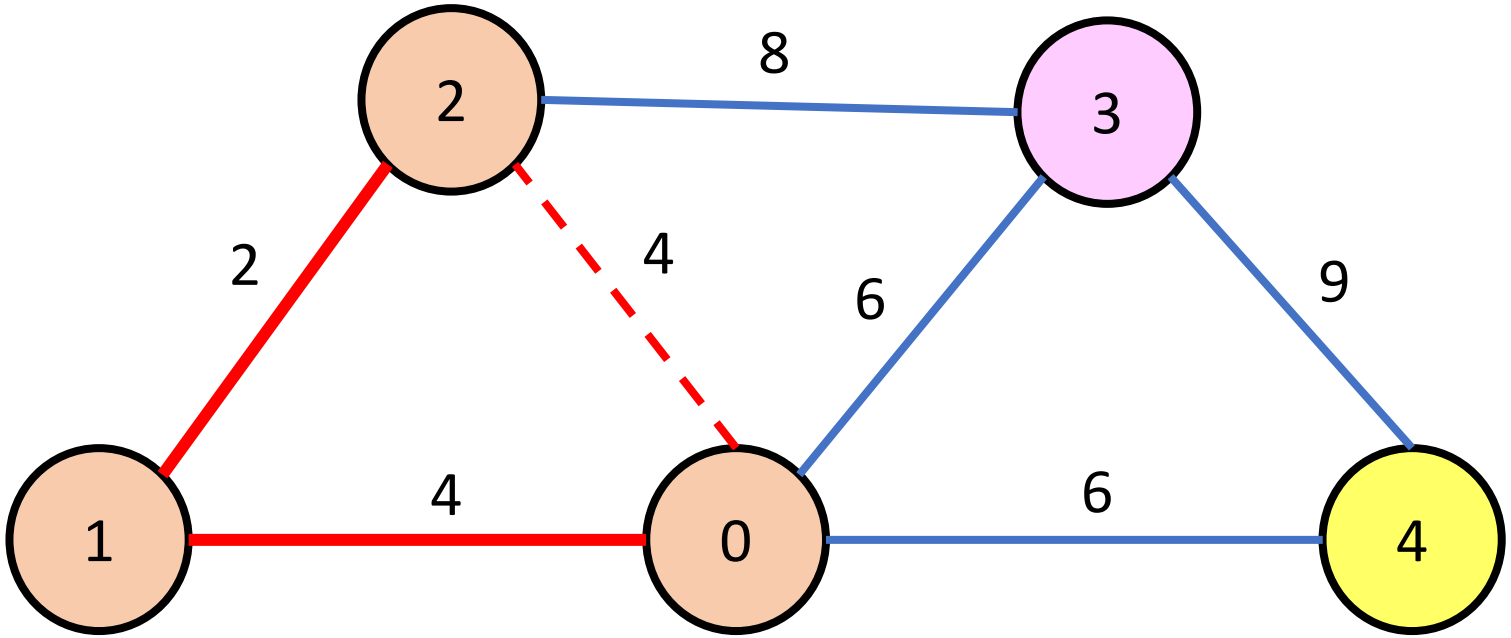
El algoritmo de Kruskal

Ejemplo



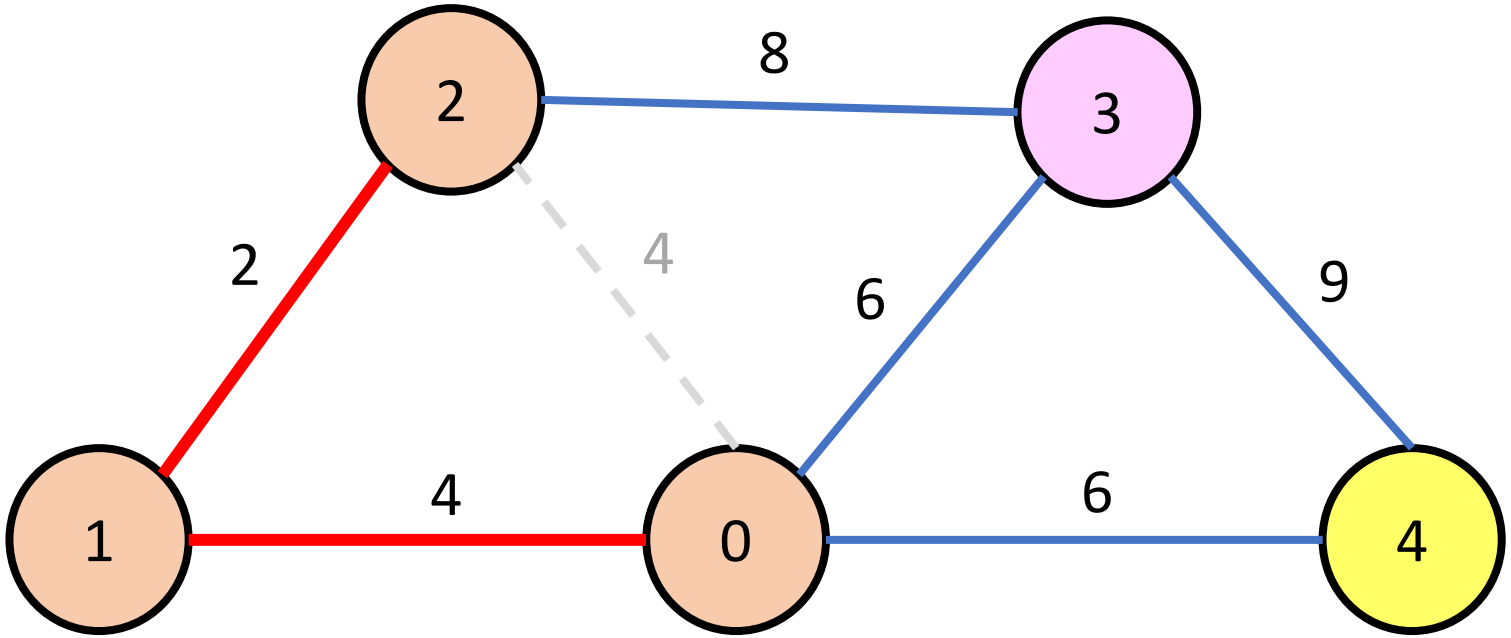
El algoritmo de Kruskal

Ejemplo



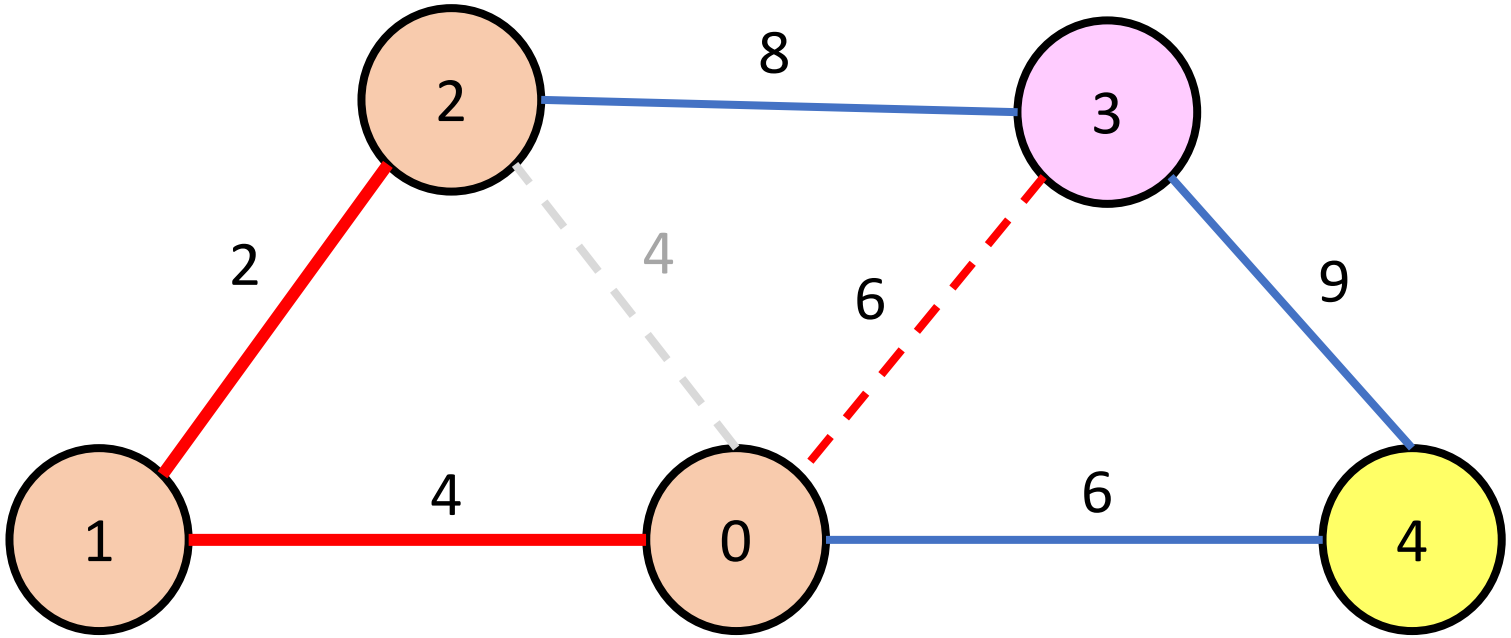
El algoritmo de Kruskal

Ejemplo



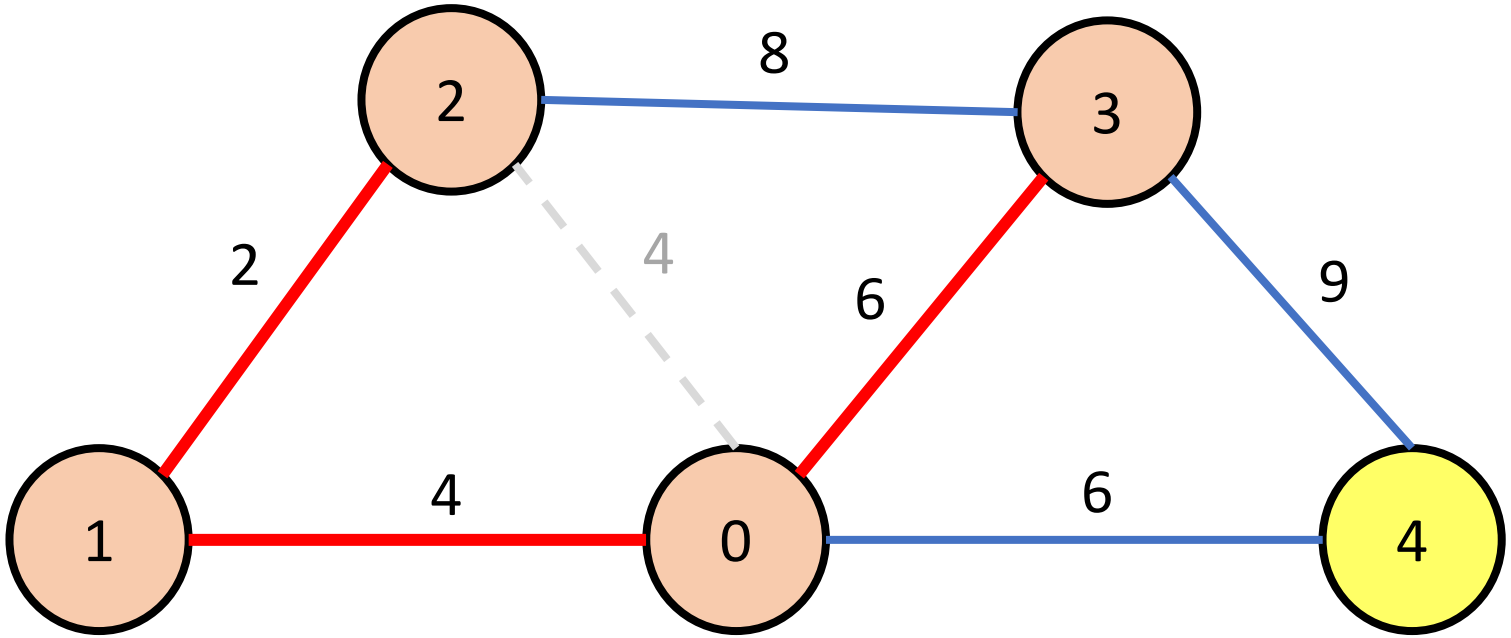
El algoritmo de Kruskal

Ejemplo



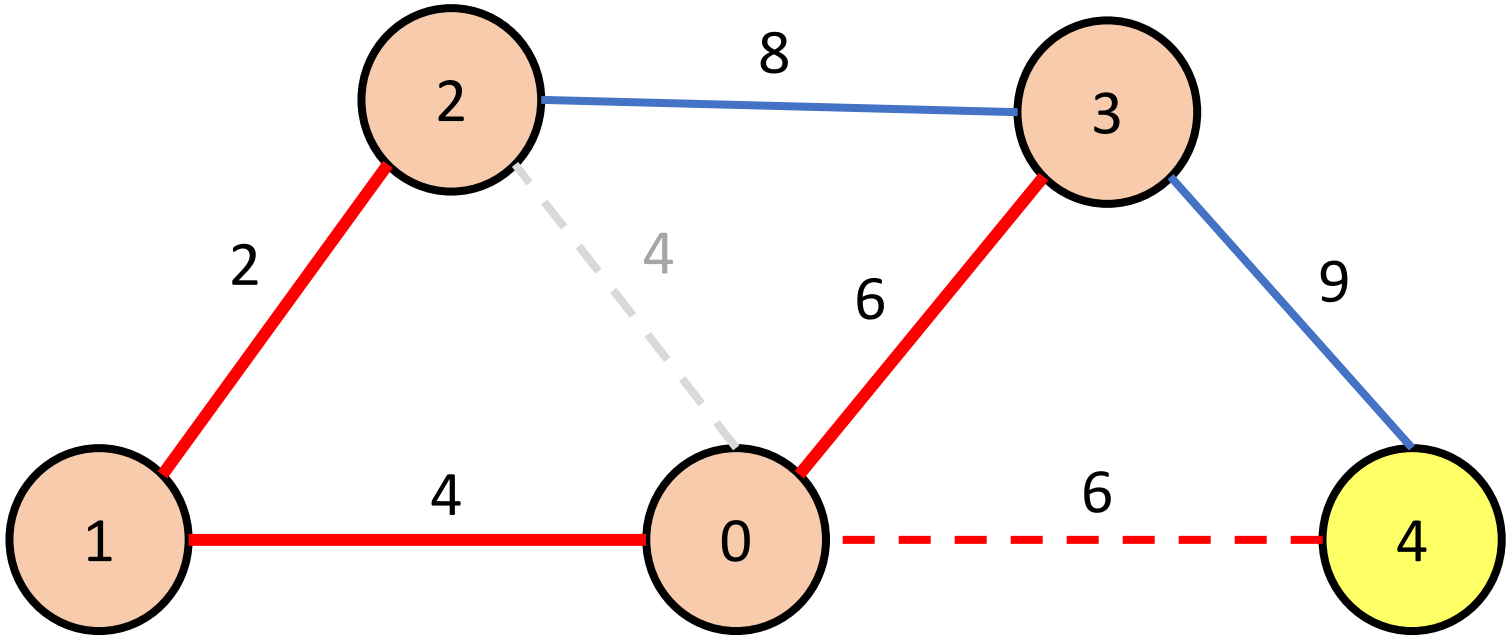
El algoritmo de Kruskal

Ejemplo



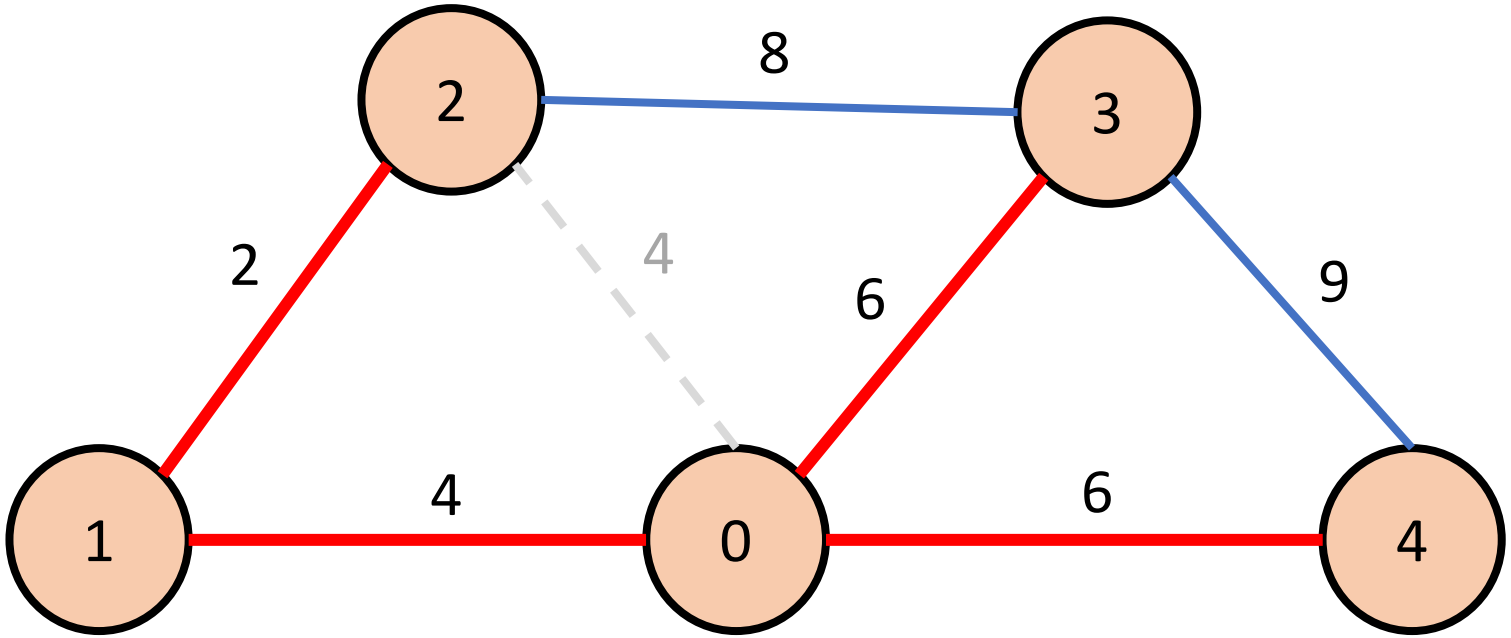
El algoritmo de Kruskal

Ejemplo



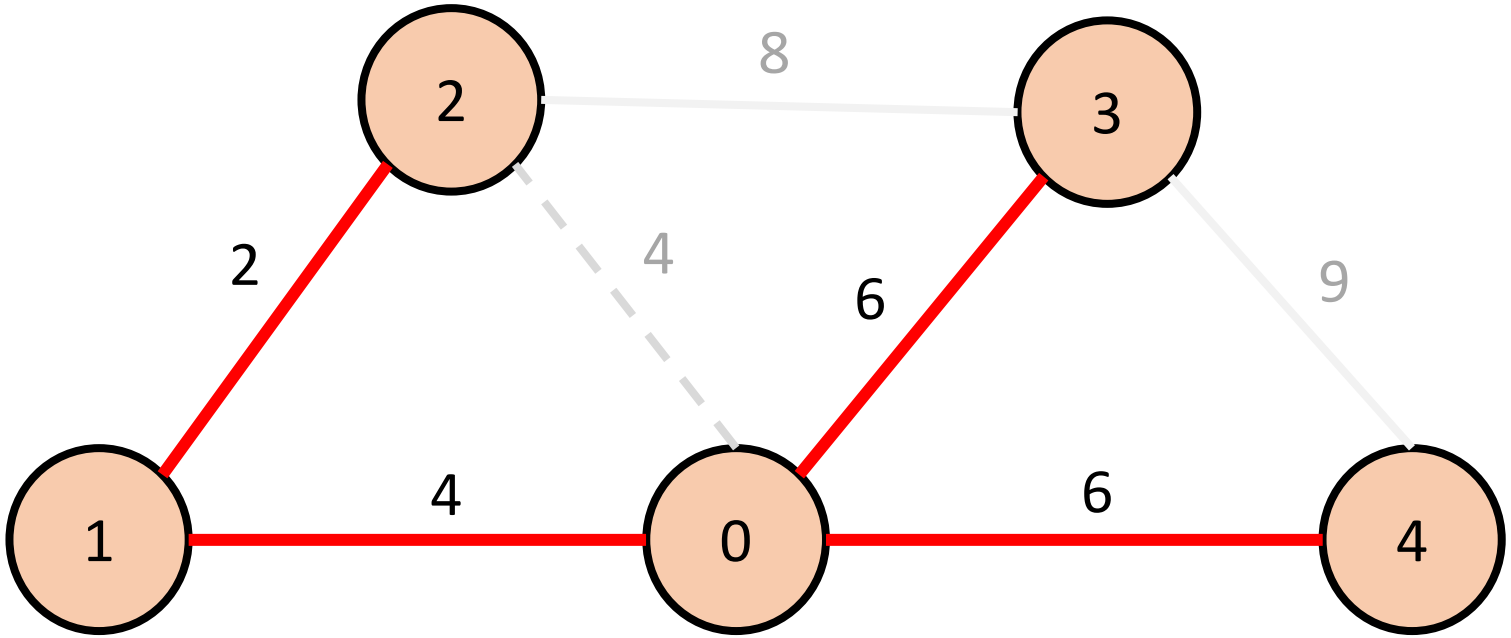
El algoritmo de Kruskal

Ejemplo



El algoritmo de Kruskal

Ejemplo



El algoritmo de Kruskal

Complejidad

Ordenar las aristas: $O(E \log E)$
Para cada arista usamos $join(x, y)$ de Union-Find: $O(E \log E)$

Complejidad: $O(E \log E)$

El algoritmo de Kruskal

Código

```cpp
ll kruskal(vector<pair<ll,pair<int,int>>> edges, int n){

    // edges -> guarda las aristas en al forma {peso, {u, v}}

    sort(edges.begin(), edges.end());       // Ordenamos las aristas de peso menor a mayor.
    unionFind dsu(n+1);
    int cntAristas = 0;     // Cuantas aristas ya hemos añadido
    ll res = 0;        // Peso total del árbol de expansión mínima.
    for(auto e : edges){
        ll peso = e.first;     // Peso de la arista actual
        int u = e.second.first;
        int v = e.second.second;
        if(dsu.join(u, v)){
            cntAristas++;
            res += peso;
        }
        if(cntAristas == n-1){      // Terminamos porque solo se necesitan n - 1 aristas para cualquier árbol de n nodos.
            return res;
        }
    }
    if(cntAristas < n - 1){
        return -1;          // No se pudo conectar todos los nodos.
    }
    return res;
}
```

Enlaces útiles

- https://cses.fi/book/book.pdf
- https://cp-algorithms.com/
- https://usaco.guide/gold/shortest-paths?lang=cpp

Contest:
https://vjudge.net/contest/630501
pass: ioi2024